OnBuch+

SIGNED COURSE • PREMIUM

COMPUTER SCIENCE • UPPER SIXTH | UPPER SIXTH — COMPUTER SCIENCE

CHAPTER 11

MANAGING MEMORY RESOURCES.

SUGGESTED TIME: 5 PERIODS

IN THIS CHAPTER

This chapter explains how the operating system manages main memory: from basic concepts and contiguous allocation to paging, segmentation and virtual memory with demand paging. Students learn to compare allocation strategies, compute address translations and evaluate page replacement algorithms — all classic GCE Advanced Level examination topics.

- By the end of this chapter I can explain the role of memory management in an operating system and define key terms (logical/physical address, relocation, protection).
- By the end of this chapter I can distinguish between compile-time, load-time and execution-time address binding.
- By the end of this chapter I can describe contiguous allocation schemes (single partition, fixed partitions, dynamic partitions) and compute internal and external fragmentation.
- By the end of this chapter I can apply the first-fit, best-fit and worst-fit placement strategies to a given memory map.
- By the end of this chapter I can explain paging and segmentation and translate a logical address into a physical address using a page table.
- By the end of this chapter I can explain the principle of virtual memory and demand paging, including page faults and their handling.

DOWNLOAD ONBUCH
Free • Google Play
AI tutor, past papers, results

GO ONBUCH+
All signed courses, unlimited AI tutor, PDF sheets — OnBuch+ tab in the app

THIS SIGNED COURSE CONTAINS

COURSE complete, illustrated, step by step

METHODS and worked examples

EXERCISES exam style + solutions

SUMMARY SHEET the essentials to remember

Signed • OnBuch+
ONBUCH+ TEACHING TEAM
FOLLOWS THE OFFICIAL MINESEC SYLLABUS

OnBuch+

CONTENTS

1. Section 1: Memory Management Basics (Lesson 51)
2. Section 2: Contiguous Memory Allocation (Lesson 52)
3. Section 3: Non-contiguous Memory Allocation — Paging and Segmentation (Lesson 53)
4. Section 4: Virtual Memory and Demand Paging (Lesson 54)
5. Integration activity
6. Methods and worked examples
7. Exercises
8. Solutions to the exercises
9. Summary sheet

CHAPTER OBJECTIVES

- By the end of this chapter I can explain the role of memory management in an operating system and define key terms (logical/physical address, relocation, protection).
- By the end of this chapter I can distinguish between compile-time, load-time and execution-time address binding.
- By the end of this chapter I can describe contiguous allocation schemes (single partition, fixed partitions, dynamic partitions) and compute internal and external fragmentation.
- By the end of this chapter I can apply the first-fit, best-fit and worst-fit placement strategies to a given memory map.
- By the end of this chapter I can explain paging and segmentation and translate a logical address into a physical address using a page table.
- By the end of this chapter I can explain the principle of virtual memory and demand paging, including page faults and their handling.
- By the end of this chapter I can simulate the FIFO, LRU and Optimal page replacement algorithms and count page faults for a reference string.
- By the end of this chapter I can compare memory management techniques and justify the choice of a technique for a given situation.

PREREQUISITES

- Basic computer architecture: role of RAM, CPU and secondary storage (Lower Sixth).
- Number systems and binary/hexadecimal representation of addresses.
- Concept of a process and multiprogramming (chapter on process management).
- Basic data structures: tables, queues and lists.
- Units of storage: byte, KB, MB, GB and conversions.

1 SECTION 1: MEMORY MANAGEMENT BASICS (LESSON 51)

At a busy cybercafé in Buea, ten students share one computer: each opens a browser, a Word document and a music player, yet the machine with only 4 GB of RAM never mixes up their files. How does the computer decide where each program lives in memory, what happens when RAM is full, and why does the machine suddenly slow down when too many applications are open? Just as the manager of a crowded market hall in Douala must allocate stalls to traders, reuse freed spaces and keep a register of who occupies what, the operating system must manage memory carefully. In this chapter we open the "memory manager's register" and discover the techniques — from simple partitions to virtual memory — that make multitasking possible on every computer and phone in Cameroon.

In this first section we lay the foundations: why memory must be managed at all, the hierarchy of storage devices, the crucial distinction between logical and physical addresses, how and when addresses are bound, and the hardware mechanisms (relocation and limit registers, the MMU, swapping) that make sharing memory safe.

1.1 Why Memory Must Be Managed

Observation. A modern computer runs multiprogramming: several processes reside in RAM at the same time, and the CPU switches rapidly between them. This is what allows you to type a document while music plays and a download continues. But RAM is **limited**: if three programs each need 2 GB and the machine has only 4 GB, they cannot all fit naively. Someone must decide:

- **where** each process is placed in RAM (allocation);
- **how much** space each one receives, and how freed space is reused (tracking and deallocation);
- **how** to stop one process from reading or overwriting another's data (protection);
- **what** to do when RAM is full (swapping, and later virtual memory).

DEFINITION **Memory management**

Memory management is the set of operating-system techniques used to keep track of which parts of main memory are free or occupied, to allocate memory to processes, to deallocate it when they finish, to protect processes from one another, and to move processes between RAM and secondary storage when necessary. The part of the OS responsible is called the **memory manager**.

KEY POINTS **The four tasks of the memory manager**

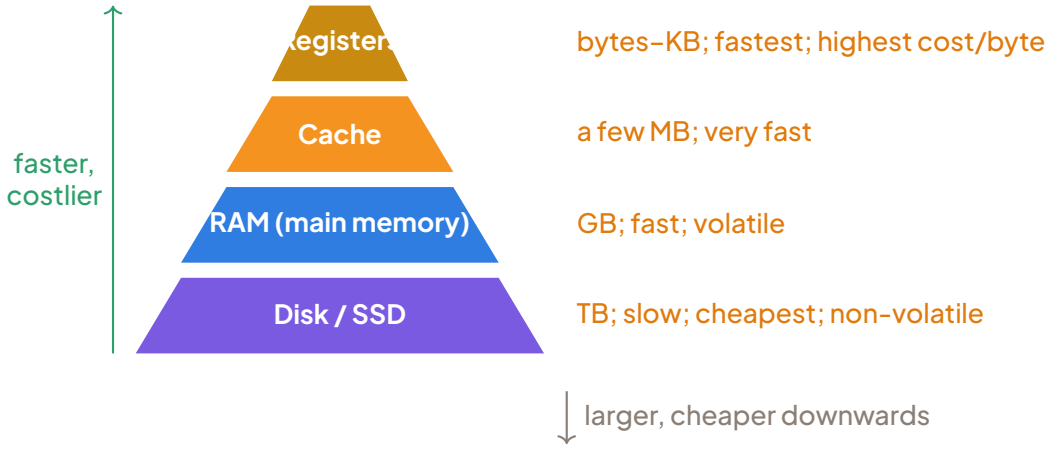
1. Keep track of free and used memory (e.g. with a memory map, bitmaps, or linked lists of free blocks).
2. Allocate memory to a process when it is created or grows.
3. Deallocate (reclaim) memory when a process terminates or shrinks.
4. Protect each process's space so that no process can corrupt another or the OS itself.

1.2 The Memory Hierarchy

Observation. Why not build the whole computer out of the fastest memory? Because speed costs money. A tiny amount of very fast memory (registers) is affordable; 4 GB of it would cost a fortune. Engineers therefore organise storage as a **hierarchy**: small, fast, expensive memories at the top; large, slow, cheap memories at the bottom. Data that the CPU needs right now is kept near the top; the rest waits lower down.

DEFINITION **Memory hierarchy**

The **memory hierarchy** is the organisation of storage into levels ordered by speed, capacity and cost per byte: registers (inside the CPU, fastest, a few bytes), cache (very fast SRAM, a few MB), main memory / RAM (DRAM, a few GB), and secondary storage (hard disk or SSD, hundreds of GB to TB, slowest, cheapest, and non-volatile).



The memory hierarchy pyramid: capacity increases downwards while speed and cost per byte increase upwards.

DID YOU KNOW? **Why the café computer slows down**

When RAM is full, the OS starts moving data between RAM and the disk. A disk access is thousands of times slower than a RAM access, so the whole machine appears to "freeze" or crawl. This is exactly what the students in Buea observe when too many applications are open — and it motivates swapping and virtual memory, studied later in this chapter.

1.3 Logical and Physical Addresses

Observation. When a program is written and compiled, the programmer has no idea where in RAM it will eventually be loaded — the free space available changes from minute to minute. So a program cannot contain real RAM addresses. Instead, each process works with its own private numbering of memory, starting from 0, as if it owned the whole machine. The translation to real RAM locations happens later.

DEFINITION **Logical address, physical address, address space**

A **logical address** (also called **virtual address**) is an address generated by the CPU as seen by the process, inside that process's own **logical address space** — the range of addresses from 0 up to the process's size minus one. A **physical address** is the real address of a byte in RAM, as seen by the memory hardware. The set of all physical addresses is the **physical address space**.

EXEMPLE **Two processes, same logical address**

Process A (a browser) and process B (a music player) both use logical address 500. A is loaded at physical address 20000, B at physical address 60000. When A accesses logical 500, the hardware delivers physical 20500; when B accesses logical 500, it delivers 60500. Same logical address, different physical bytes — neither process can disturb the other.

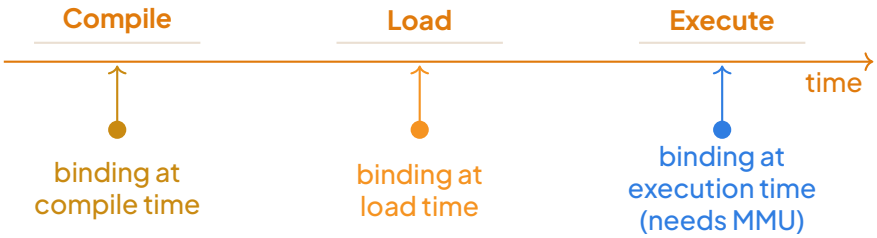
1.4 Address Binding: Compile Time, Load Time, Execution Time

Observation. At some moment, the symbolic names and logical addresses in a program must be bound to physical addresses. The question is: when? Three answers are possible, each with consequences.

DEFINITION **Address binding**

Address binding is the mapping of instructions and data of a program to physical memory addresses. It can occur at three moments:

- Compile time: if the future location in RAM is known in advance, the compiler generates absolute code with fixed physical addresses. If the location changes, the program must be recompiled.
- Load time: the compiler produces relocatable code; the loader converts logical addresses to physical ones when the program is loaded. Once loaded, the program cannot be moved.
- Execution time: binding is delayed until the program runs; every address is translated by hardware on each memory access. The process can be moved in RAM during its execution. This requires special hardware: the **Memory Management Unit (MMU)**.



The later the binding, the more flexible the memory management.

The three moments of address binding along the life of a program.

KEY POINTS **Execution-time binding needs hardware**

Execution-time binding — the scheme used by all modern operating systems — is impossible in software alone: checking and translating every address by program would be far too slow. It requires the **Memory Management Unit (MMU)**, a hardware component sitting between the CPU and RAM (often integrated into the CPU chip).

1.5 Relocation and the Role of the MMU

Observation. Suppose the browser is loaded at physical address 20000. Its instruction "load the word at logical address 500" must become "load physical address 20500". Rather than rewriting the program, the hardware simply adds the starting address to every logical address. This is relocation.

DEFINITION **Relocation**

Relocation is the process of adjusting a program's addresses so that it can run at a physical location different from the one assumed when it was written. In static relocation, ad-

dresses are adjusted once, at load time. In dynamic relocation, addresses are adjusted during execution by the MMU, which adds the content of the **base register** (also called **relocation register**) — holding the physical start address of the process — to every logical address:

$$\text{Physical Address} = \text{Logical Address} + \text{Base Register}.$$

EXEMPLE **Dynamic relocation in numbers**

Process P occupies 8000 bytes starting at physical address 14000, so the base register holds 14000. The CPU issues logical address 1234. The MMU computes $1234 + 14000 = 15234$ and sends 15234 to RAM. If the OS later moves P to address 50000, it simply writes 50000 into the base register — the program itself is not modified at all.

1.6 Memory Protection: The Limit Register

Observation. Adding a base register is not enough: what stops a faulty or malicious program from issuing a huge logical address and landing inside another process's memory? We also need to know where the process's space ends.

DEFINITION **Memory protection with base and limit registers**

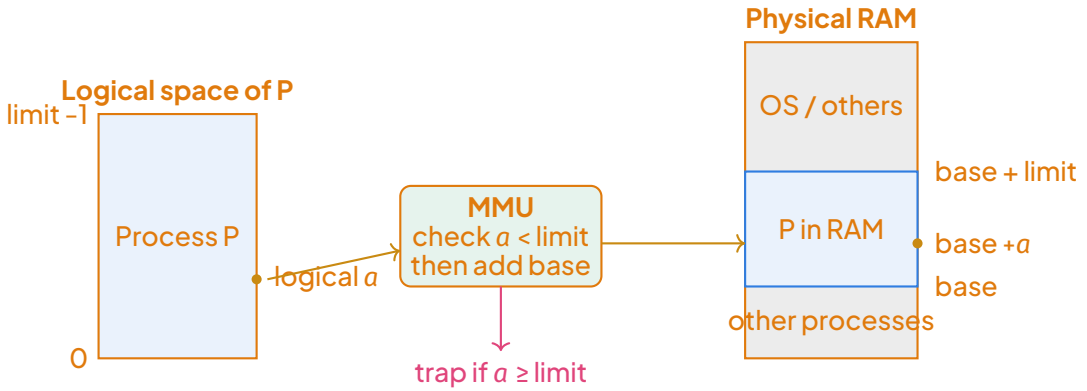
Memory protection is the mechanism that prevents a process from accessing memory outside its own address space. With the base/limit scheme, the MMU uses two registers: the **base register** (start of the process in RAM) and the **limit register** (size of the process's logical address space). Every logical address a generated by the CPU is checked:

- if $0 \le a < \text{limit}$, the access is legal: physical address $= a + \text{base}$;
- if $a \ge \text{limit}$, the access is illegal: the MMU raises a **trap** (a hardware interrupt) and the OS terminates or suspends the offending process.

METHOD **How the MMU translates an address and detects an illegal access**

1. The CPU generates a logical address a (e.g. from an instruction fetch or a data access).
2. The MMU compares a with the limit register.
3. If $a \ge \text{limit}$, the MMU raises a trap (addressing error); the OS takes control and typically kills the process with an error message.

4. Otherwise, the MMU adds the base register: physical address = a + base.
5. The physical address is placed on the memory bus and RAM performs the access.



The MMU maps logical addresses into physical RAM using the base register and blocks out-of-range accesses using the limit register.

WORKED EXAMPLE 1 **Applying the base/limit mechanism**

A process is loaded at physical address 30000 and its size is 12000 bytes. The base register therefore holds 30000 and the limit register 12000. For each logical address generated by the CPU, give the physical address or state what happens: (a) 0; (b) 4500; (c) 11999; (d) 12000; (e) 25000.

SOLUTION

Apply the rule: legal if a < 12000; physical = a + 30000.

- (a) a = 0: legal (0 < 12000). Physical = 0 + 30000 = 30000.
- (b) a = 4500: legal. Physical = 4500 + 30000 = 34500.
- (c) a = 11999: legal (the last valid byte). Physical = 11999 + 30000 = 41999.
- (d) a = 12000: illegal because 12000 ≥ limit. The MMU raises a trap; the OS intervenes.
- (e) a = 25000: illegal (25000 ≥ 12000). Trap; no memory access occurs, so no other process is corrupted.

Note the boundary: with limit = 12000, valid logical addresses run from 0 to 11999 inclusive — exactly 12000 bytes.

WARNING **Confusing logical and physical addresses**

A very common GCE mistake is to mix up the two spaces. Remember: the CPU (and the program) only ever sees logical addresses; RAM only ever sees physical addresses; the MMU converts one into the other. In examination questions, always ask yourself: "who produced this address?" If it comes from the program/CPU, it is logical and must be checked against the limit and added to the base before touching RAM. Also, never add the base to an illegal address: the limit check comes first.

1.7 Swapping

Observation. Back to our cybercafé: when RAM is completely full and a new process needs space, what can the memory manager do? One simple answer is to move a whole inactive process out of RAM onto the disk, freeing its space, and bring it back later.

DEFINITION **Swapping**

Swapping is the operation of temporarily copying an entire process from RAM to a reserved area of secondary storage (the backing store or swap area) — this is swapping out (roll out) — and later copying it back into RAM when it is needed again — swapping in (roll in). Swapping frees memory for other processes but is slow, because disk access is far slower than RAM access.

EXEMPLE **Swapping and dynamic relocation**

Process P is swapped out from physical address 14000. Later there is no free block at 14000, but space exists at 60000. Thanks to dynamic relocation, P can be swapped back in at 60000: the OS simply sets the base register to 60000. With static (load-time) relocation this would be impossible without reloading and re-adjusting the whole program — one more reason modern systems bind addresses at execution time.

WARNING **Swapping moves whole processes**

Do not confuse swapping with paging (Section 4): classic swapping transfers an entire process between RAM and disk. Transferring small fixed-size pieces of a process on demand is a different, finer technique (demand paging) studied later. Also remember the cost: frequent swapping is the main cause of the slowdown observed when too many applications are open.

1.8 Summary and Consolidation

KEY POINTS **Key points of Lesson 51**

- Multiprogramming puts several processes in limited RAM, so the OS must track, allocate, deallocate and protect memory.
- The memory hierarchy trades speed against capacity and cost: registers > cache > RAM > disk.
- Programs use logical addresses; RAM uses physical addresses; address binding links the two at compile time, load time or execution time.
- Execution-time binding requires the MMU and enables dynamic relocation: physical = logical + base.
- Protection uses the limit register: any logical address ≥ limit causes a trap.
- Swapping rolls a whole process out to disk and back to free RAM, at the cost of slow disk transfers.

EXERCISE 1 **Checking your understanding**

1. Explain in your own words why multiprogramming makes memory management necessary.
2. A process has base register 50000 and limit register 9000. For each logical address, give the physical address or the system's reaction: 100; 8999; 9000; 45000.
3. Why does execution-time binding require hardware support, while compile-time binding does not?
4. A computer becomes very slow when many applications are opened. Using the notions of this lesson, explain what is happening.
5. State two advantages of dynamic relocation over static relocation.

SOLUTION **Exercise 1**

1. With several processes resident in RAM at once, memory is shared and scarce: the OS must decide where each process goes, reuse freed space, prevent processes from interfering with each other, and cope when RAM is full.
2. Rule: legal if $a < 9000$; physical = $a + 50000$. So 100 → 50100; 8999 → 58999 (last

valid byte); 9000 → illegal, trap; 45000 → illegal, trap.

3. With execution-time binding, every memory access must be checked and translated. Doing this in software would make each access many times slower; only dedicated hardware (the MMU) can perform the limit check and base addition at full speed. Compile-time binding is done once, before execution, so no hardware is needed at run time.
4. RAM is full, so the OS swaps whole processes (or parts of them) between RAM and disk. Disk accesses are thousands of times slower than RAM accesses, so the machine spends most of its time transferring data instead of executing programs: the system crawls.
5. A process can be moved in RAM during execution (e.g. swapped back in at a different address) simply by changing the base register; and the OS can reorganise memory without reloading or modifying programs. With static relocation, addresses are fixed at load time and the process cannot be moved.

We now understand why memory is managed and the hardware that makes it safe. The next section studies the first concrete allocation strategy: contiguous memory allocation, where each process receives one single block of RAM.

2 SECTION 2: CONTIGUOUS MEMORY ALLOCATION (LESSON 52)

In Section 1 we saw that the memory manager must decide where each process will live in main memory, and that it must keep track of which parts of memory are occupied and which are free. The very first allocation schemes ever designed all shared one simple idea: give each process **one single continuous block** of memory. This family of techniques is called **contiguous memory allocation**. It is historically important, easy to understand, and still examined at the GCE Advanced Level because it introduces the two great enemies of memory management: internal and external fragmentation.

2.1 The Principle of Contiguous Allocation

DEFINITION **Contiguous Memory Allocation**

A memory allocation scheme is said to be **contiguous** when every process must be loaded into one single, unbroken (continuous) block of main memory. A process of size n bytes can only be loaded if there exists a free region of at least n consecutive bytes.

Think of a car park where every lorry must park in one straight line of consecutive spaces: a lorry needing 4 spaces cannot use 2 spaces here and 2 spaces there, even if 4 spaces are free in

total. Exactly the same constraint applies to processes in contiguous allocation. This single rule is the source of both the simplicity and the weaknesses of the whole family.

2.2 Single Contiguous Allocation

The simplest possible scheme was used by early monitors and simple batch systems: memory is divided into just **two regions** — one for the **operating system** (usually at low addresses, together with the interrupt vector) and one for **a single user process**.

- Only **one user program** runs at a time; there is no multiprogramming.
- If the program is smaller than the user region, the rest of memory is simply **wasted**.
- Protection is minimal: a **limit register** may be used to stop the user program from overwriting the operating system.

This scheme is easy to implement but makes poor use of both the memory and the CPU (while the single process waits for input/output, the processor is idle). To allow multiprogramming, memory must be shared between several processes — which leads to partitioned allocation.

2.3 Fixed (Static) Partitioning

DEFINITION **Fixed Partitioning**

In **fixed (static) partitioning**, main memory is divided, once and for all at system start-up, into a fixed number of partitions. Each partition holds at most one process. Partitions may be of equal size or of unequal sizes. The operating system keeps a **partition table** recording, for each partition: its number, its size, its base address and its status (free or occupied, and by which process).

With **equal-size partitions**, any process larger than one partition cannot run at all, and small processes waste the rest of their partition. **Unequal partitions** reduce this waste: small jobs go to small partitions, large jobs to large ones. A process is placed either in any free partition that is big enough, or in the smallest free partition that can hold it (to waste less space).

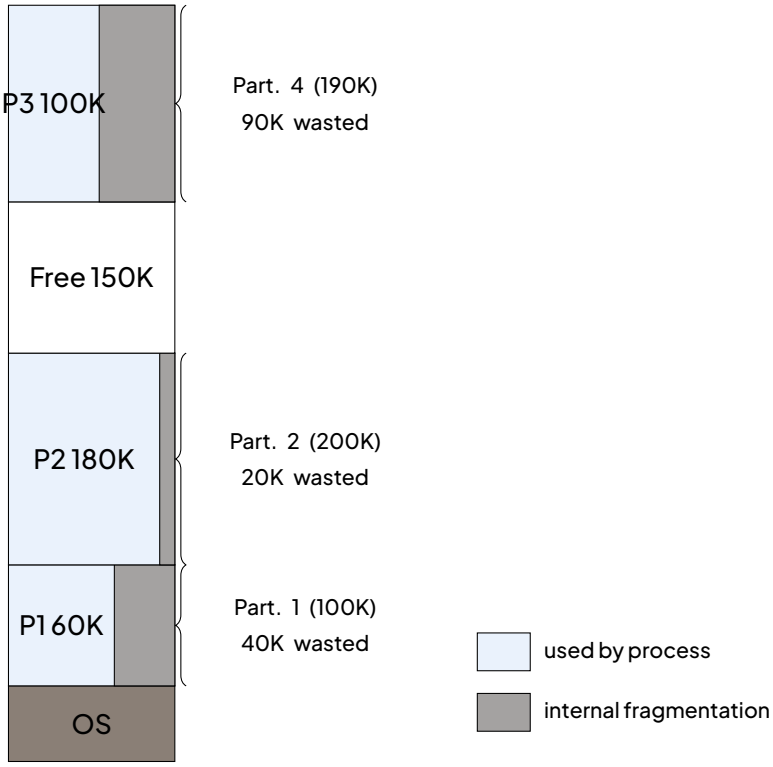
EXEMPLE **A partition table**

Memory of 640 KB (after the OS) divided into four fixed partitions:

Partition	Size	Status	Process
1	100 KB	Used	P1 (60 KB)
2	200 KB	Used	P2 (180 KB)
3	150 KB	Free	—
4	190 KB	Used	P3 (100 KB)

P1 wastes 100 - 60 = 40 KB inside partition 1; P2 wastes 200 - 180 = 20 KB inside partition

2; P3 wastes 190 − 100 = 90 KB inside partition 4. This wasted space is inside an allocated partition: it is internal fragmentation.



Fixed partitions of unequal sizes. The shaded parts are wasted space inside allocated partitions: internal fragmentation.

2.4 Internal and External Fragmentation

DEFINITION **Internal vs External Fragmentation**

- **Internal fragmentation**: memory is wasted inside an allocated block, because the block given to a process is larger than the process itself. Typical of fixed partitioning.
- **External fragmentation**: memory is wasted between allocated blocks, in the form of many small free holes, none of which is individually large enough for a waiting process — even though their total would be. Typical of dynamic partitioning.

WARNING **Do not confuse the two fragmentations**

A frequent exam mistake is to swap the two terms. Remember: internal = inside an allocated block (fixed partitions); external = outside, in the free space between blocks (dynamic partitions). Also note that a free fixed partition is not "external fragmentation": it is simply an unused partition.

2.5 Dynamic (Variable) Partitioning

Fixed partitioning wastes space because partition boundaries are decided before knowing the processes. The natural improvement: create the partition when the process arrives, with exactly its size.

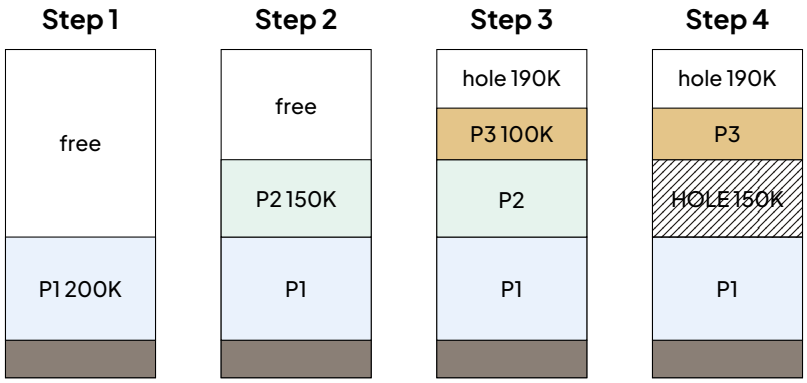
DEFINITION **Dynamic Partitioning**

In **dynamic (variable) partitioning**, partitions are created at load time: when a process of size n is loaded, the memory manager carves out a block of exactly n bytes from a free region. There is initially no internal fragmentation. However, as processes are loaded and terminated, free memory becomes split into scattered **holes**: external fragmentation appears.

EXEMPLE **Birth of a hole — step by step**

Memory of 640 KB (after the OS). Three processes arrive: P1 (200 KB), P2 (150 KB), P3 (100 KB). Then P2 terminates.

- Step 1: P1 loaded at the bottom: $[0, 200)$.
- Step 2: P2 loaded next: $[200, 350)$.
- Step 3: P3 loaded next: $[350, 450)$. Free: $[450, 640)$, one hole of 190 KB.
- Step 4: P2 terminates: a hole of 150 KB appears at $[200, 350)$, between two busy blocks. We now have two holes (150 KB and 190 KB) totalling 340 KB — yet a new process of 300 KB cannot be loaded, because no single hole is big enough. That is external fragmentation in action.



Dynamic partitioning: P1, P2, P3 are loaded; P2 terminates and leaves a hole trapped between P1 and P3 (external fragmentation).

2.6 Placement Strategies: First-Fit, Best-Fit, Worst-Fit

When several holes are free, which one should receive the new process? Three classic strategies exist.

DEFINITION **Placement Strategies**

- **First-fit**: scan memory from the beginning and place the process in the first hole large enough. Fast; tends to fill low memory and leave holes at high addresses.
- **Best-fit**: place the process in the smallest hole that is large enough. Wastes least space on this allocation, but leaves very small, often unusable leftover holes.
- **Worst-fit**: place the process in the largest available hole, hoping the leftover remains big enough to be useful.

METHOD **Applying a placement strategy in an exam**

1. Draw the memory map: list all holes with their sizes in memory order (top to bottom).
2. For each arriving process, scan the holes according to the strategy:
   - First-fit: first hole with size ≥ process size, in address order.
   - Best-fit: hole with size ≥ process size whose size is minimal.
   - Worst-fit: hole with size ≥ process size whose size is maximal.
3. Subtract the process size from the chosen hole; write the leftover hole back on the map, in the same position.

4. If no hole is large enough, the process waits (or compaction is needed) — say so explicitly.
5. Repeat for each process, updating the map at every step.

WORKED EXAMPLE 2 **First-fit, best-fit, worst-fit on the same data**

Free holes, in memory order: 100 KB, 500 KB, 200 KB, 300 KB, 600 KB. Processes arrive in order: P1 = 212 KB, P2 = 417 KB, P3 = 112 KB, P4 = 426 KB. Place them with each strategy.

SOLUTION

First-fit (first hole big enough, in address order):

- P1 (212) → hole 500; leftover 288. Holes: 100, 288, 200, 300, 600.
- P2 (417) → hole 600; leftover 183. Holes: 100, 288, 200, 300, 183.
- P3 (112) → hole 288; leftover 176. Holes: 100, 176, 200, 300, 183.
- P4 (426): no hole ≥ 426 ⇒ P4 must wait.

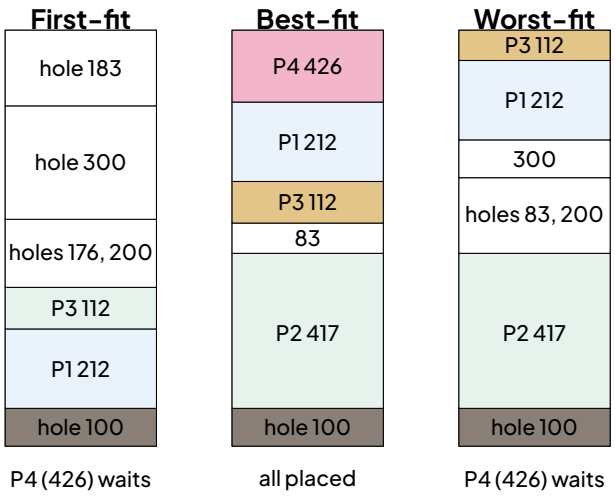
Best-fit (smallest sufficient hole):

- P1 (212) → hole 300; leftover 88. Holes: 100, 500, 200, 88, 600.
- P2 (417) → hole 500; leftover 83. Holes: 100, 83, 200, 88, 600.
- P3 (112) → hole 200; leftover 88. Holes: 100, 83, 88, 88, 600.
- P4 (426) → hole 600; leftover 174. All four processes placed!

Worst-fit (largest hole):

- P1 (212) → hole 600; leftover 388. Holes: 100, 500, 200, 300, 388.
- P2 (417) → hole 500; leftover 83. Holes: 100, 83, 200, 300, 388.
- P3 (112) → hole 388; leftover 276. Holes: 100, 83, 200, 300, 276.
- P4 (426): largest hole is 300 < 426 ⇒ P4 must wait.

Conclusion: on this example, only best-fit places all four processes. No strategy is always the winner — it depends on the data.



The same holes (100, 500, 200, 300, 600 KB) and the same requests (212, 417, 112, 426 KB) handled by the three strategies. Only best-fit places P4 here.

WARNING **Best-fit is not always "best"**

The name is a trap! **Best-fit** minimises waste for the current allocation only. Because it always picks the tightest hole, it leaves behind tiny leftover holes (a few KB) that no real process can ever use — external fragmentation in its most irritating form. First-fit, although simpler, often performs comparably or better in practice and is much faster. In the exam, never claim a strategy is "the best" in general: justify with the given numbers.

2.7 Keeping Track of Free Space: Bit Maps and Linked Lists

The memory manager must maintain a data structure describing which parts of memory are free.

Bit maps. Memory is divided into small fixed-size **allocation units** (e.g. 4 KB). A **bit map** is an array of bits, one per unit: 0 = free, 1 = occupied. For 256 MB of memory with 4 KB units, we need $2^{28}/2^{12} = 2^{16} = 65\,536$ bits = 8 KB — cheap. Finding a free block of k units means finding a run of k consecutive zeros, which can be slow.

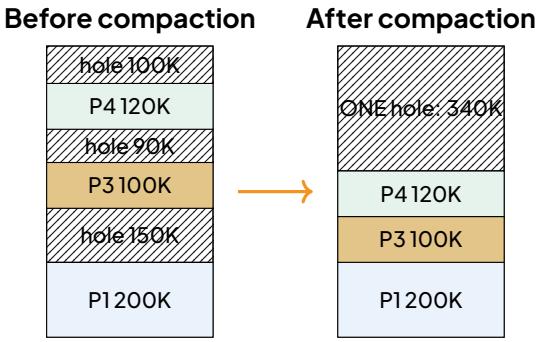
Linked lists. A linked list alternates entries describing each hole (H) and each process (P) segment, with start address and length: e.g. H(0,100) – P(100,200) – H(300,150) – … Allocation is fast to record, and when a process terminates, its entry is **merged with neighbouring holes** (**coalescing**) so that adjacent free space forms one big hole. Depending on whether the neighbours of the freed block are processes or holes, four cases arise: the freed block may stand alone as a new hole, merge with the hole above, merge with the hole below, or merge with both holes into one large hole.

2.8 Compaction

DEFINITION **Compaction**

Compaction is the operation of moving processes in memory so that all the small scattered holes are gathered into one single large hole. It eliminates external fragmentation, but it is only possible if the system supports **dynamic relocation**: the hardware (a relocation/base register added to every logical address, with a limit register for protection) must allow a process to run correctly after being moved, at execution time.

Compaction is **costly**: a simple scheme that moves everything toward low addresses may copy many megabytes and stops all user processes during the move. Systems therefore compact only when necessary — typically when a process is waiting although total free memory is sufficient.



Compaction: processes are slid toward low addresses; three holes (150 + 90 + 100 KB) merge into a single 340 KB hole. Requires dynamic relocation hardware.

2.9 Advantages and Limits of Contiguous Allocation

KEY POINTS **Summary: contiguous allocation**

Advantages:

- Simple to understand and to implement; low overhead per process.
- Fast address translation (one base register) and fast access (no indirection).
- Fixed partitioning makes multiprogramming possible with minimal hardware.

Limits:

- Internal fragmentation with fixed partitions; external fragmentation with dynamic partitions.
- A process can be refused although total free memory is sufficient.
- Compaction is expensive and needs relocation hardware.

- The maximum process size is limited by the largest single free block.

Because main memory kept growing while programs grew even faster, wasting memory in fragments became unacceptable. Modern systems therefore break the "one continuous block" rule and use non-contiguous allocation — **paging** and **segmentation** — which we study in Section 3.

WARNING **Exam tip**

In any allocation question, always draw the memory map and update it after every event (arrival or departure of a process). Write hole sizes explicitly, keep holes in address order, and state clearly when a process must wait. Most marks are lost by students who try to do everything in their head and mix up the leftover holes.

EXERCISE 2 **Placement strategies**

Free holes in memory order: 200 KB, 400 KB, 100 KB, 500 KB. Processes arrive: P1 = 250 KB, P2 = 350 KB, P3 = 90 KB, P4 = 380 KB. For each strategy (first-fit, best-fit, worst-fit), give the final memory map and state which processes, if any, must wait.

SOLUTION **Exercise 1**

First-fit: P1 → 400 (left 150); P2 → 500 (left 150); P3 → 200 (left 110); P4: holes are 110, 150, 100, 150 — all < 380, so P4 waits. Best-fit: P1 → 400 (left 150); P2 → 500 (left 150); P3 → 100 (left 10); P4: largest hole is 200 < 380, so P4 waits. Worst-fit: P1 → 500 (left 250); P2 → 400 (left 50); P3 → 250 (left 160); P4: holes are 200, 50, 100, 160 — P4 waits. Here no strategy places P4; only compaction (total free = 200 + 50 + 100 + 160 = 510 KB, gathered into one block) could help.

3 SECTION 3: NON-CONTIGUOUS MEMORY ALLOCATION — PAGING AND SEGMENTATION (LESSON 53)

In the previous section we studied contiguous memory allocation, where each process occupies a single continuous block of physical memory. We saw that this leads to **external fragmentation** (free memory broken into small, non-contiguous holes) and forces the operating system to use compaction or restrictive placement strategies (first-fit, best-fit, worst-fit).

Non-contiguous allocation removes the requirement that a process be loaded into one contin-

uous block. Instead, a process is divided into smaller pieces that can be placed in any available free frames of physical memory. This completely eliminates external fragmentation. The two main non-contiguous schemes are **paging** (fixed-size pieces) and **segmentation** (variable-size pieces that correspond to logical units of the program). Modern systems often combine them (**paged segmentation**).

3.1 The Idea of Non-contiguous Allocation

DEFINITION **Non-contiguous Allocation**

A memory allocation scheme in which the different parts of a process (code, data, stack, heap) are placed in separate, non-adjacent blocks of physical memory. The operating system maintains a mapping from the process's logical address space to the scattered physical frames.

KEY POINTS **Why non-contiguous?**

- Eliminates external fragmentation: any free frame can satisfy a request; no need for a single large hole.
- Supports virtual memory: only the needed pieces must be resident (demand paging, Section 4).
- Enables sharing and protection: individual pages or segments can be shared between processes or protected (read-only, execute-only).

3.2 Paging

Physical and logical division.

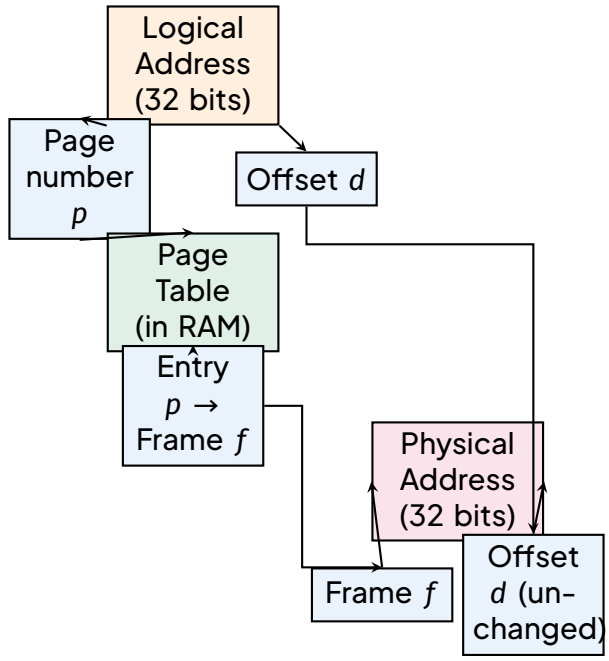
Physical memory is divided into fixed-size blocks called **frames** (or **page frames**). The logical address space of a process is divided into blocks of the same size called **pages**. Page size is always a power of two (typically 4 KB to 16 KB on modern systems, 4 KB being the most common).

DEFINITION **Page, Frame, Page Table, Offset**

- **Frame**: A fixed-size block of physical memory.
- **Page**: A fixed-size block of logical memory (same size as a frame).
- **Page Table**: A per-process data structure that maps each page number to a frame number. It is stored in main memory; its base address is held in the Page Table Base Register (PTBR) (also called Page Directory Base Register or CR3 on x86).

- **Offset (d)**: The displacement within a page/frame. Because page size = frame size = 2^k bytes, the offset uses exactly k bits and is identical in the logical and physical addresses.

Numeric example:
Page size = 4 KB = 2^{12} B ⇒ offset = 12 bits.
Logical address = `0x00003A7C` (binary: `0000 0000 0000 0000 0011 1010 0111 1100`).
Split: p = `0000 0000 0000 0000 0011` = 3, d = `1010 0111 1100` = `0xA7C`.
Page table entry 3 → frame f = 5.
Physical address = (5 ≪ 12) + `0xA7C` = `0x00005A7C`.



Paging address translation: logical address (p, d) → page table → physical address (f, d). The offset d is copied unchanged.

METHOD **Translating a Logical Address to a Physical Address (Paging)**

Given: page size = 2^k bytes, logical address LA (usually in hexadecimal or decimal), and the page table.

1. Determine k: $k = \log_2$(page size in bytes). The offset d occupies the k least-significant bits.
2. Split LA: Write LA in binary (or hexadecimal). The k LSBs are the offset d; the remaining MSBs are the page number p.
3. Look up frame: Use p as index into the page table to obtain the frame number f. (If p is out of range or the valid bit is 0 → **page fault**.)
4. Form physical address: $PA = (f \times 2^k) + d$. In binary: concatenate the binary representation of f (as many bits as needed for physical memory size) with the k bits of d.

WORKED EXAMPLE 3 **Paging Address Translation**

Statement: A system uses paging with a page size of 2 KB (2^{11} bytes). Physical memory has 64 KB (16-bit physical addresses). The page table for a process is:

Page p	Frame f	Valid
0	4	1
1	7	1
2	1	1
3	0	0

Translate the following logical addresses (given in hexadecimal) to physical addresses, or indicate a page fault:

1. `0x0A7C`
2. `0x1800`
3. `0x0C00`

SOLUTION

Detailed solution:

- Page size = 2 KB = 2^{11} B ⇒ k = 11 bits for offset.
- Logical address width: Not given, but page numbers in table go up to 3 ⇒ at least 2 bits for page number. We'll just split the given hex numbers.

1. LA = `0x0A7C`

$$\texttt{0x0A7C} = \texttt{0000 1010 0111 1100}_2 \quad \text{(16 bits for clarity)}$$

$$\text{Offset } d = 11 \text{ LSBs} = \texttt{010 0111 1100}_2 = \texttt{0x27C}$$

$$\text{Page number } p = \text{remaining MSBs} = \texttt{0000 }1_2 = 1$$

$$\text{Page table: } p = 1 \rightarrow f = 7 \text{ (valid)}$$

$$PA = (7 \times 2^{11}) + \texttt{0x27C} = 7 \times \texttt{0x800} + \texttt{0x27C} = \texttt{0x3800} + \texttt{0x27C} = \texttt{0x3A7C}$$

Physical address = `0x3A7C`.

2. LA = `0x1800`

$$\texttt{0x1800} = \texttt{0001 1000 0000 0000}_2$$

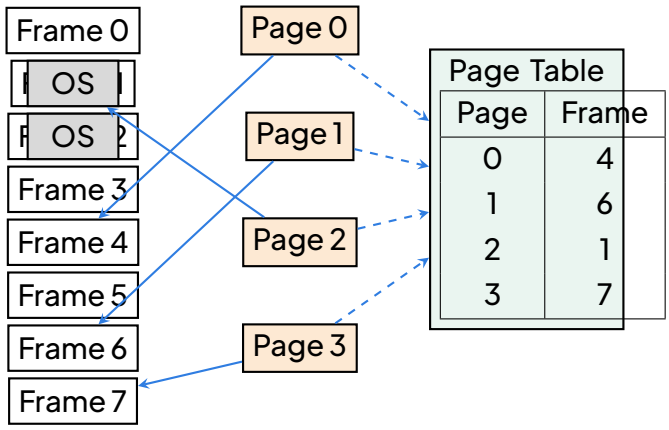
$$d = \texttt{000 0000 0000}_2 = 0$$

$$p = \texttt{0001 }1_2 = 3$$

$$\text{Page table: } p = 3 \rightarrow \text{Valid} = 0 \quad \Rightarrow \quad \text{Page Fault}$$

3. *LA* = 0x0C00

$$0x0C00 = 0000\ \ 1100\ \ 0000\ \ 0000_2$$
$$d = 000\ \ 0000\ \ 0000_2 = 0$$
$$p = 0001\ \ 1_2 = 3 \quad \text{(same as above)}$$
$$\Rightarrow \text{Page Fault}$$



Pages of a single process scattered across non-contiguous physical frames. The page table records the mapping.

PROPERTY **Fragmentation in Paging**

- External fragmentation: Eliminated. Any free frame can be used; no need for contiguous holes.
- Internal fragmentation: Exists. The last page of a process is rarely completely full. On average, half a page per process is wasted. With page size S, average internal fragmentation $\approx S/2$ per process.
- Page size trade-off:
  - Small pages → less internal fragmentation, but larger page tables (more entries) and more page faults (less locality per page).
  - Large pages → smaller page tables, better I/O transfer efficiency, but more internal fragmentation and poorer resolution for sharing/protection.

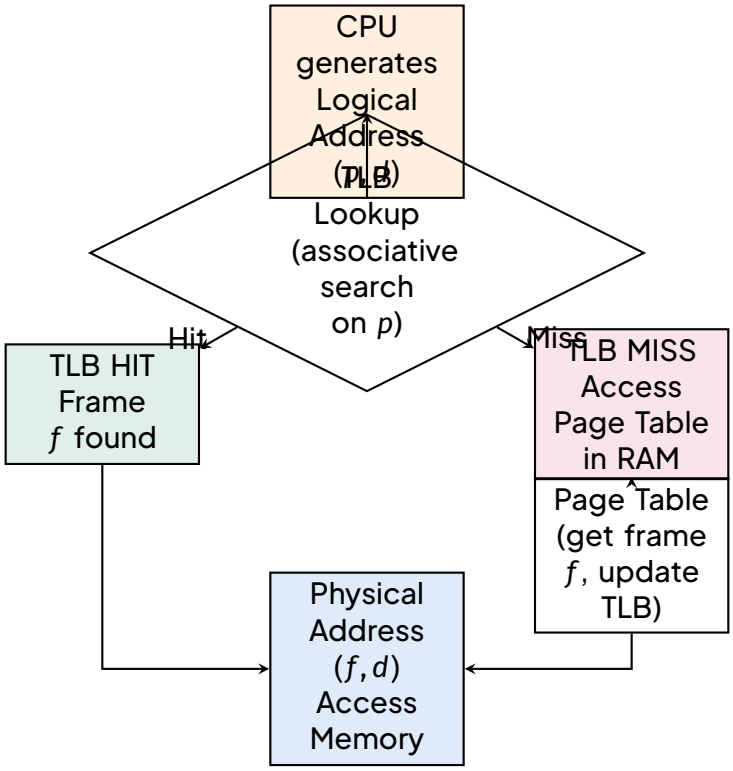
Typical compromise: 4 KB (x86, ARM), 16 KB or 64 KB on some RISC systems.

WARNING **Offset is Copied Unchanged**

In paging, only the page number is translated. The offset d is identical in the logical and physical addresses. A common mistake in exams is to modify the offset or to misalign the split when the page size is not a power of two (but page size is always a power of two in real systems).

Translation Lookaside Buffer (TLB).

Every memory access in paging requires two memory accesses: one to read the page table entry, one to access the actual data. This would halve performance. The **TLB** is a small, fast, associative hardware cache (part of the MMU) that stores recently used page-table entries.



TLB lookup flow: hit (fast path, 1 memory access) vs miss (slow path, 2 memory accesses).

KEY POINTS **Effective Access Time (EAT) — GCE useful extra**

Let t_{TLB} = TLB access time, t_{mem} = memory access time, h = TLB hit ratio.

$$\begin{aligned} EAT &= h \times (t_{TLB} + t_{mem}) + (1 - h) \times (t_{TLB} + 2t_{mem}) \\ &= t_{TLB} + t_{mem} + (1 - h)t_{mem} \end{aligned}$$

Typical values: $t_{TLB} \approx 10$ ns, $t_{mem} \approx 100$ ns, $h > 98\% \Rightarrow$ EAT $\approx$ 110–120 ns (close to single

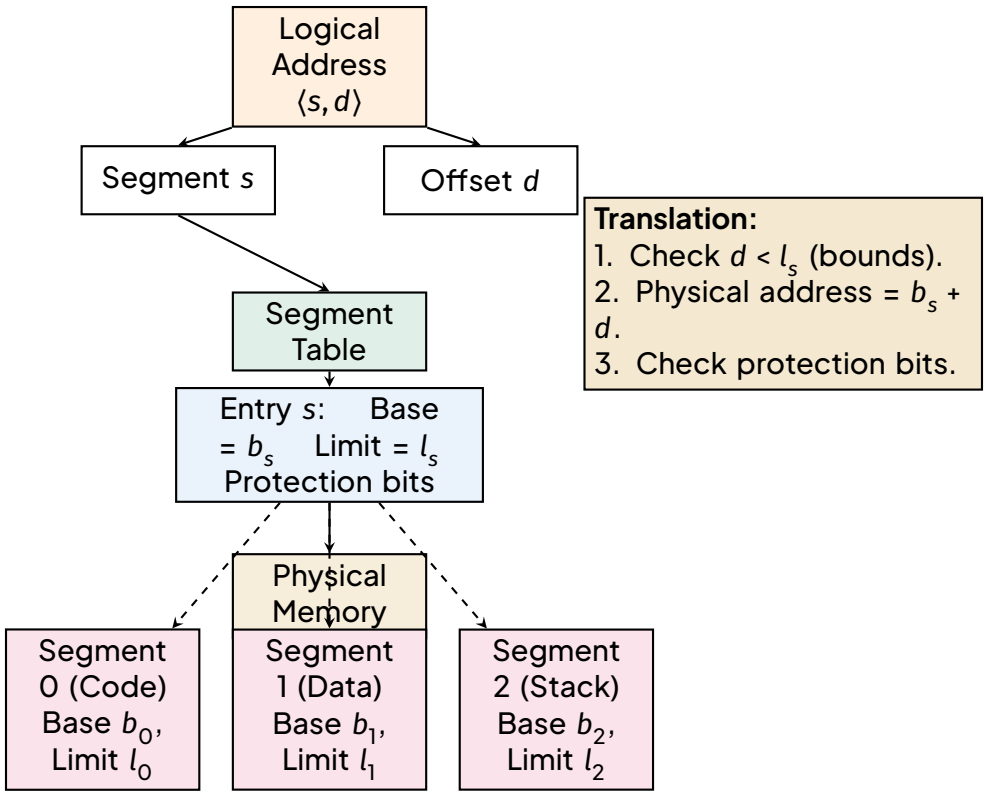
memory access).

3.3 Segmentation

Segmentation reflects the programmer's view of memory: a program is a collection of logical units (code, data, stack, heap, libraries) of varying sizes. Each unit is a **segment**.

DEFINITION **Segment, Segment Table, Base, Limit**

- **Segment**: A variable-length logical unit (e.g., main program, function, array, stack). Identified by a segment number s.
- **Segment Table**: Per-process table; each entry contains:
  - **Base**: Starting physical address of the segment in memory.
  - **Limit**: Length of the segment (for bounds checking).
  - Protection bits (read/write/execute), valid bit, etc.
- Logical address: A pair $\langle s, d \rangle$ where d is the offset within segment s.



Segmentation: logical address $\langle s, d \rangle$ mapped via segment table (base, limit) to physical address. Segments can be placed non-contiguously.

METHOD **Segmentation Address Translation**

Given a segment table with base b_s and limit l_s for each segment s, and a logical address $\langle s, d \rangle$:

1. Use s to index the segment table. If s is invalid → **segmentation fault**.
2. Check bounds: if $d \geq l_s$ → segmentation fault (trap to OS).
3. Check protection: if access type (read/write/execute) not allowed → protection fault.
4. Physical address $PA = b_s + d$.

WORKED EXAMPLE 4 **Segmentation Translation and Faults**

Statement: A process has the following segment table:

Segment s	Base	Limit	Protection
0 (Code)	1000	500	R, X
1 (Data)	4000	1200	R, W
2 (Stack)	8000	300	R, W

Determine the physical address or the fault for each logical address $\langle s, d \rangle$:

1. $\langle 0, 200 \rangle$
2. $\langle 1, 1500 \rangle$
3. $\langle 2, 250 \rangle$ (write access)
4. $\langle 3, 100 \rangle$

SOLUTION

Solution:

1. $s = 0$: base=1000, limit=500. $d = 200 < 500$ OK. Protection allows read/execute. $PA = 1000 + 200 = 1200$.
2. $s = 1$: base=4000, limit=1200. $d = 1500 \geq 1200 \rightarrow$ Bounds violation (segmentation fault).
3. $s = 2$: base=8000, limit=300. $d = 250 < 300$ OK. Write access allowed (R,W). $PA = 8000 + 250 = 8250$.
4. $s = 3$: no entry in table → Invalid segment (segmentation fault).

KEY POINTS **Sharing and Protection in Segmentation**

Because segments correspond to logical units, sharing is natural: two processes can share the same code segment (read-only, execute) by having segment-table entries pointing to the same physical base with appropriate protection bits. Each process keeps its own private data/stack segments. This is harder with pure paging (requires page-level sharing and same page alignment).

3.4 Paged Segmentation (Overview)

Pure segmentation suffers from **external fragmentation** (variable-size segments leave holes). Pure paging has no external fragmentation but does not match the logical structure. Modern OSs (Linux, Windows, macOS on x86-64) use **paged segmentation**:

- The logical address space is divided into **segments** (code, data, stack, etc.).
- **Each segment is divided into fixed-size pages.**

- The segment table entry points to a **page table** for that segment (instead of a base/limit).
- Logical address → $\langle s, p, d \rangle$: segment s, page p within segment, offset d.
- Translation: segment table → page table base → page table → frame f → physical address (f, d).

This combines the logical benefits of segmentation (sharing, protection per logical unit) with the physical memory management benefits of paging (no external fragmentation, simple frame allocation).

3.5 Comparison of Memory Allocation Schemes

Criterion	Contiguous	Paging	Segmentation
Allocation unit	One contiguous block per process	Fixed-size pages/frames	Variable-size segments
External fragmentation	**Yes** (major problem)	**No**	**Yes** (variable sizes)
Internal fragmentation	No (if exact fit) or small	**Yes** (avg $S/2$ per process)	No (segment fits exactly)
Address translation	Simple (base + limit)	Page table (page → frame)	Segment table (base + limit)
Sharing	Difficult (whole process)	Possible per page (needs same alignment)	**Natural per logical unit**
Protection	Whole process (or base/limit)	Per page (R/W/X)	**Per segment (R/W/X)**
Hardware support	Base/limit registers	MMU with page table walk, TLB	MMU with segment table, bounds check
Programmer's view	Single linear array	Invisible (hardware-managed)	**Matches logical structure**

WARNING **Common GCE Traps**

- Confusing page/frame numbers with addresses: The page table maps page number → frame number. The physical address is (frame × page size) + offset.
- Mis-splitting the address: Always compute $k = \log_2(\text{page size})$. The offset is the k least significant bits. The page number is the rest. If page size = 4 KB (2^{12}), offset = 12 bits.
- Forgetting the offset is unchanged: In both paging and segmentation, the offset d is added to the base/frame unchanged. Only the high-order bits change.
- Segmentation bounds check: The condition is $d <$ limit (or $d \leq$ limit − 1). $d =$ limit is already out of bounds.

- TLB vs Page Table: TLB is a cache for page table entries. A TLB miss triggers a page table walk. The page table is always in memory (unless swapped out in advanced VM).

DID YOU KNOW? **Cameroon Context: Memory in Embedded Systems**

Many embedded systems used in agriculture (e.g., soil moisture sensor nodes in the Far North), health (portable diagnostic devices in Yaoundé hospitals), or finance (POS terminals in Douala markets) run on microcontrollers (STM32, ESP32) with no MMU. They use flat contiguous memory or simple MPU (Memory Protection Unit) regions (similar to segmentation but static). Understanding paging/segmentation helps you appreciate why Linux/Android (on smartphones) can run many apps safely while a bare-metal sensor node runs a single firmware image.

EXERCISE 3 **Practice Problems**

1. A system uses paging with page size 8 KB. Physical memory is 1 MB. The page table for a process has 128 entries. How many bits in the logical address for (a) page number, (b) offset, (c) total logical address bits? How many bits in the physical address for frame number and offset?
2. Given page size 4 KB and the page table: $0 \to 5, 1 \to 2, 2 \to 7, 3 \to 1$ (all valid). Translate logical addresses: `0x1234`, `0x3000`, `0x0FFF`.
3. A segmentation system has segment 0: base=2000, limit=1000, protection=RW; segment 1: base=5000, limit=500, protection=RX. What happens for $\langle 0, 999 \rangle$ (write), $\langle 1, 500 \rangle$ (read), $\langle 1, 499 \rangle$ (execute), $\langle 2, 0 \rangle$?
4. Explain why paging eliminates external fragmentation but segmentation does not. Give a concrete example with a memory map.
5. (GCE-style) A process has 4 pages. Page size = 2 KB. The page table is stored in memory at physical address `0x8000`. Each page table entry is 4 bytes. The PTBR holds `0x8000`. If the CPU generates logical address `0x2A50`, show the complete translation steps including the physical address of the page table entry accessed.

SOLUTION **Exercise 1**

1. Page size = 8 KB = 2^{13} B $\Rightarrow$ offset = 13 bits. Physical memory = 1 MB = 2^{20} B $\Rightarrow$ physical address = 20 bits. Number of frames = $2^{20}/2^{13} = 2^7 = 128$ frames $\Rightarrow$ frame number = 7 bits. Page table has 128 entries $\Rightarrow$ page number = 7 bits. Logical address = page bits + offset

bits = 7 + 13 = 20 bits. Physical address = frame bits + offset bits = 7 + 13 = 20 bits.
2. Page size = 4 KB = 2^{12} ⇒ offset = 12 bits (3 hex digits).

- `0x1234`: p = `0x1` = 1, d = `0x234`. f = 2. PA = (2 ≪ 12) + `0x234` = `0x2000` + `0x234` = `0x2234`.
- `0x3000`: p = `0x3` = 3, d = `0x000`. f = 1. PA = `0x1000`.
- `0x0FFF`: p = `0x0` = 0, d = `0xFFF`. f = 5. PA = `0x5000` + `0xFFF` = `0x5FFF`.

3.

- ⟨0, 999⟩: d = 999 < 1000, write allowed (RW). PA = 2000 + 999 = 2999.
- ⟨1, 500⟩: d = 500 ≮ 500 → bounds fault.
- ⟨1, 499⟩: d = 499 < 500, execute allowed (RX). PA = 5000 + 499 = 5499.
- ⟨2, 0⟩: segment 2 not in table → invalid segment fault.

4. Paging uses fixed-size frames; any free frame fits any page → no external holes. Segmentation uses variable-size segments; a 10 KB segment cannot fit in a 9 KB hole even if total free memory > 10 KB → external fragmentation. Example: RAM = 100 KB. Segments A=30KB, B=40KB, C=20KB loaded. B terminates → hole 40KB. New segment D=35KB fits. D terminates → hole 35KB adjacent to 40KB? If not contiguous, two holes 35KB and 40KB. Request for 50KB fails despite 75KB free.
5. Page size = 2 KB = 2^{11} ⇒ offset = 11 bits. LA = `0x2A50` = `0010 1010 0101 0000`$_2$. p = `0010` 1_2 = 5 (but process has only 4 pages 0–3 → page fault!). If we assume page table has entry for 5: PTBR = `0x8000`. Entry size = 4 bytes. Page table entry address = PTBR + p × 4 = `0x8000` + 5 × 4 = `0x8014`. Read entry at `0x8014` → frame f. PA = (f ≪ 11) + `0x250`.

4 SECTION 4: VIRTUAL MEMORY AND DEMAND PAGING (LESSON 54)

In Section 3 we saw how paging and segmentation allow a process to occupy non-contiguous regions of physical memory. But one strong assumption remained: the whole process had to be loaded into RAM before it could execute. In this final section we remove that assumption. We shall see how the operating system can run programs that are larger than the physical memory itself, by keeping only the needed pages in RAM and the rest on disk. This is the idea of **virtual memory**, and its engine is **demand paging**.

4.1 Virtual Memory: Motivation and Principle

EXPERIMENT **A concrete observation**

Open a large application on a modest machine: a video editor, a big spreadsheet, or an

IDE with many plugins. The program may occupy several gigabytes on disk, yet the computer may have only 4 GB of RAM, shared with the operating system and other programs. How can it run at all? Observation shows that at any given moment, a program uses only a small part of its code and data: the menu you have opened, the function currently executing, the data currently displayed. The rest sleeps on disk. The operating system exploits exactly this fact.

DEFINITION **Virtual memory**

Virtual memory is a memory management technique that allows the execution of processes that are not completely in physical memory. It separates the **logical memory** seen by the programmer (a large, uniform address space) from the **physical memory** actually available (RAM). The logical address space can therefore be much larger than physical RAM; the parts of a process that are not currently needed are kept on a fast secondary storage area (the swap space or page file on disk).

KEY POINTS **Benefits of virtual memory**

- A program can be larger than physical RAM and still execute.
- Only part of each program needs to be in RAM, so more programs can reside in memory at once: the **degree of multiprogramming** increases, and CPU utilisation improves.
- Programmers are freed from worrying about physical memory limits; they write code against a large, simple logical address space.
- Less I/O is needed to load or swap programs, so each program starts faster.

4.2 Demand Paging and the Valid/Invalid Bit

DEFINITION **Demand paging**

Demand paging is the strategy of loading a page into physical memory only when it is actually needed (on demand), i.e. when the running process first references an address belonging to that page. Pages that are never referenced are never loaded, saving memory and disk I/O.

How does the hardware know whether a page is in RAM or still on disk? We extend each **page table entry** with one extra bit, the **valid/invalid bit**:

- **Valid (v)**: the page is legal and currently loaded in a physical frame; the entry gives the frame number.
- **Invalid (i)**: either the page does not belong to the process's logical address space (illegal reference ⇒ the process is terminated), or the page is legal but currently **on disk** (this triggers a page fault).

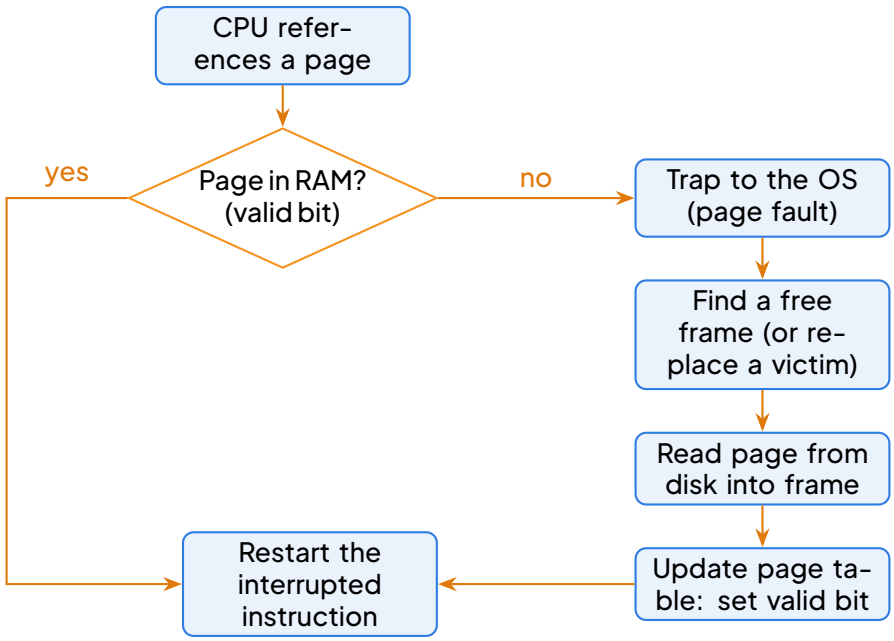
4.3 Page Faults and Their Handling

DEFINITION **Page fault**

A **page fault** is a hardware trap raised by the memory management unit (MMU) when a process references a page whose page-table entry is marked invalid because the page is not currently in physical memory. It is not an error: it is a normal signal asking the operating system to bring the page into RAM.

METHOD **Steps of page-fault handling**

1. The CPU generates a logical address; the MMU consults the page table and finds the entry invalid ⇒ a trap is sent to the operating system.
2. The OS checks an internal table to decide whether the reference is legal. If illegal, the process is aborted. If legal, the page is simply not in memory.
3. The OS finds a free frame (taken from the free-frame list, or obtained by page replacement if none is free).
4. The OS schedules a disk read to copy the required page from the swap space into the free frame. While the disk works, the CPU may run another process.
5. When the disk transfer finishes, the OS updates the page table (frame number, bit set to valid) and its internal tables.
6. The OS restarts the interrupted instruction, which now finds the page in memory and proceeds normally.



Page-fault handling: the operating system fetches the missing page from disk, updates the page table, and restarts the instruction.

4.4 Performance: Effective Access Time

A page fault is extremely expensive because it involves a **disk access** (milliseconds) instead of a simple RAM access (nanoseconds) — a ratio of roughly 10^4 to 10^5. Let p be the **page-fault rate** ($0 \le p \le 1$), m the normal memory access time, and T_{pf} the page-fault service time. The **effective access time** (EAT) is:

PROPERTY **Effective access time**

$$\text{EAT} = (1 - p) \times m + p \times T_{pf}$$

The closer p is to 0, the closer the performance is to pure RAM speed. Even a tiny page-fault rate degrades performance dramatically because $T_{pf} \gg m$. (The formula itself is examinable as a direct application.)

WORKED EXAMPLE 5 **Computing the effective access time**

Statement. Memory access time is $m = 100$ ns. A page fault costs $T_{pf} = 8$ ms $= 8\,000\,000$ ns. Compute the EAT for a page-fault rate $p = 0.001$ (one fault per 1000 accesses).

SOLUTION

Solution.

$$\begin{aligned} \text{EAT} &= (1 - 0.001) \times 100 + 0.001 \times 8\,000\,000 \\ &= 0.999 \times 100 + 8\,000 \\ &= 99.9 + 8\,000 = 8\,099.9 \text{ ns} \end{aligned}$$

The effective access time is about 81 times slower than normal RAM access, although faults occur only once per 1000 accesses. Conclusion: the page-fault rate must be kept extremely low.

4.5 Page Replacement and the Dirty Bit

When a page fault occurs and **no free frame** exists, the OS must choose a **victim page** already in memory and evict it to make room. This is **page replacement**. A second bit per page-table entry, the **dirty bit** (modify bit), records whether the page has been written to since it was loaded:

- **Clean page** (dirty bit = 0): the copy on disk is still up to date; the frame can simply be overwritten — cheap.
- **Dirty page** (dirty bit = 1): the page must first be **written back to disk** before the frame is reused — the fault then costs two disk transfers.

A good replacement algorithm therefore prefers victims that are unlikely to be used again soon, and clean pages over dirty ones.

4.6 Page Replacement Algorithms

METHOD **Simulating FIFO, LRU and OPT**

1. Draw one column per reference of the reference string, and one row per frame.
2. Process references left to right; write the frame contents after each reference.
3. If the referenced page is already in a frame, it is a hit: tick it and copy the previous column unchanged.
4. Otherwise it is a fault: if a frame is free, load the page; if not, evict the victim chosen by the algorithm (FIFO: oldest loaded; LRU: unused for the longest time; OPT: not used for the longest time in the future).
5. Count the faults and compute the fault rate $= \dfrac{\text{faults}}{\text{number of references}}$.

EXEMPLE **The three algorithms on one reference string**

Reference string: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2 with 3 frames. × = page fault, ✓= hit.
FIFO (evict the page loaded earliest) — 15 faults out of 20 with 3 frames on the classic string; on our string:

Ref	7	0	1	2	0	3	0	4	2	3	0	3	2
F1	7	7	7	2	2	2	2	4	4	4	0	0	0
F2		0	0	0	0	3	3	3	2	2	2	2	2
F3			1	1	1	1	0	0	0	3	3	3	3
	×	×	×	×	✓	×	×	×	×	×	×	✓	✓

FIFO faults = 10.
LRU (evict the page not used for the longest time):

Ref	7	0	1	2	0	3	0	4	2	3	0	3	2
F1	7	7	7	2	2	2	2	4	4	4	0	0	0
F2		0	0	0	0	0	0	0	0	3	3	3	3
F3			1	1	1	3	3	3	2	2	2	2	2
	×	×	×	×	✓	×	✓	×	×	×	×	✓	✓

LRU faults = 9.
OPT (evict the page used furthest in the future — theoretical, needs future knowledge):

Ref	7	0	1	2	0	3	0	4	2	3	0	3	2
F1	7	7	7	2	2	2	2	4	4	4	0	0	0
F2		0	0	0	0	0	0	0	0	0	0	0	0
F3			1	1	1	3	3	3	3	3	3	3	3
	×	×	×	×	✓	×	✓	×	✓	✓	×	✓	✓

OPT faults = 7. As expected: OPT ≤ LRU ≤ FIFO on this string.

WARNING **LRU is not FIFO!**

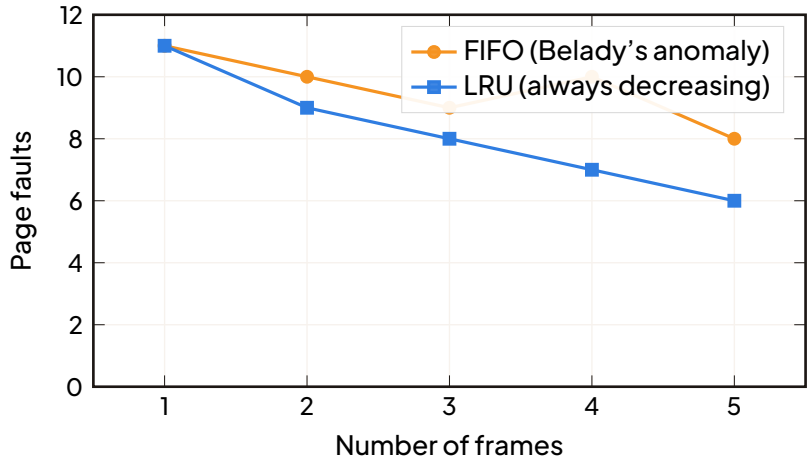
LRU replaces the page that has been unused for the longest time (look at the last reference time of each page in the string), not the page loaded earliest. A page loaded long ago but referenced recently must not be evicted under LRU. Confusing LRU with FIFO is the most frequent examination error.

WARNING **Exam tip: avoid counting errors**

In simulations, always write the frames column by column, copy the previous column on a hit, and tick every hit immediately. Count the faults only at the end, from the × marks. Never try to simulate several references in your head at once.

PROPERTY **Belady's anomaly**

Intuition suggests that adding frames can only reduce page faults. This is true for OPT and LRU (stack algorithms), but false for FIFO: for certain reference strings, increasing the number of frames increases the number of page faults. This phenomenon is called **Belady's anomaly**. Classic example: the string 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 gives 9 faults with 3 frames but 10 faults with 4 frames under FIFO. (The statement is examinable; the proof by counter-example above suffices.)



Page faults versus number of frames. FIFO may fault more with 4 frames than with 3 (Belady's anomaly), while LRU always improves with more frames.

4.7 Thrashing and Locality of Reference

DEFINITION **Thrashing**

Thrashing is the situation in which a process spends more time paging (transferring pages between disk and RAM) than executing. It occurs when a process does not have enough frames to hold the pages it actively uses: every few instructions cause a page fault, the

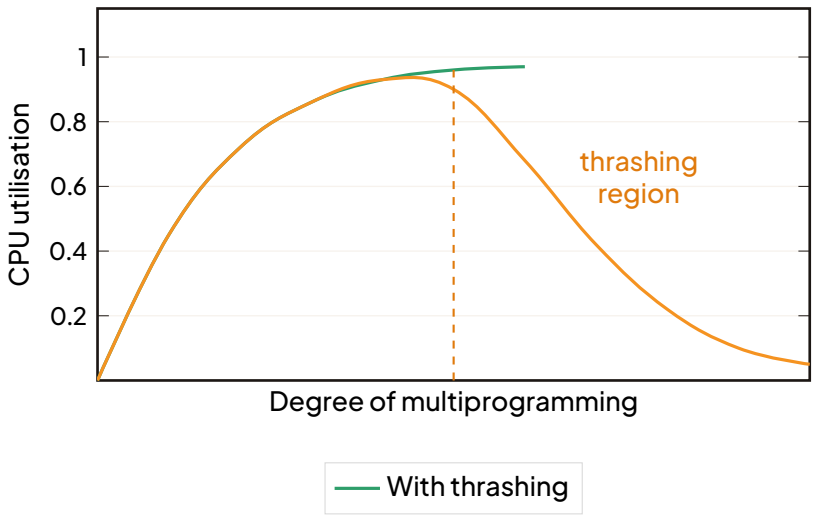
disk is saturated, and CPU utilisation collapses.

Why does it happen? If the OS sees a low CPU utilisation and naively increases the degree of multiprogramming, each process receives fewer frames, faults multiply, the CPU utilisation falls further, and the system enters a vicious circle. Remedies:

- **Reduce the degree of multiprogramming** (suspend some processes) so that survivors get enough frames.
- Use the **working set** idea: give each process at least as many frames as the set of pages it has referenced in a recent time window (its working set). If the sum of all working sets exceeds the number of frames, some process must be swapped out.

DEFINITION **Locality of reference**

The **principle of locality** states that during any short interval of execution, a process references only a small, clustered subset of its pages: instructions are executed in loops and sequentially (temporal and spatial locality of code), and data are accessed in nearby regions (arrays, stacks). This is precisely why virtual memory works: the few pages needed at any moment fit in RAM, and page faults remain rare.



CPU utilisation versus degree of multiprogramming. Beyond the optimal point, too many processes share too few frames: page faults explode and the CPU idles waiting for the disk.

4.8 Summary of Memory Management Techniques

KEY POINTS **Chapter synthesis**

Technique	Idea	Weakness
Contiguous allocation	Each process in one continuous block (fixed or variable partitions)	External fragmentation; size limits
Paging	Process split into fixed-size pages, frames anywhere	Internal fragmentation; page table cost
Segmentation	Variable-size logical units (code, data, stack)	External fragmentation
Virtual memory / demand paging	Only needed pages in RAM, rest on disk	Page faults costly; risk of thrashing

Each technique answers the weaknesses of the previous one; demand paging, guided by the locality of reference and controlled by good replacement algorithms (LRU rather than FIFO), is what makes modern multiprogrammed systems efficient.

EXERCISE 4 **Page replacement practice**

Using the reference string 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5:

1. Simulate FIFO with 3 frames and count the page faults.
2. Simulate FIFO with 4 frames and count the page faults.
3. Which phenomenon do you observe? Name it.
4. Simulate LRU with 3 frames. Is the same phenomenon possible with LRU? Justify.

SOLUTION **Exercise 1**

1. FIFO, 3 frames: faults on 1, 2, 3, 4 (evict 1), 1 (evict 2), 2 (evict 3), 5 (evict 4), then 1, 2 are hits, 3 (evict 1), 4 (evict 2), 5 hit ⇒ 9 faults. 2. FIFO, 4 frames: faults on 1, 2, 3, 4; 1, 2 hits; 5 (evict 1); 1 (evict 2); 2 (evict 3); 3 (evict 4); 4 (evict 5); 5 (evict 1) ⇒ 10 faults. 3. More frames gave more faults: this is **Belady's anomaly**. 4. LRU with 3 frames gives 10 faults here, but the anomaly is impossible with LRU: LRU is a stack algorithm, so the set of pages kept with n frames is always a subset of those kept with $n + 1$ frames; adding frames can never increase the number of faults.

5 INTEGRATION ACTIVITY

5.1 Aims of the activity

This integration activity closes the chapter by asking you to act as the memory manager of a real machine. Working in groups, you will mobilise everything studied in Lessons 51 to 54:

- distinguish **logical** and **physical addresses** and explain the role of the **MMU**;
- apply **contiguous allocation** with the **first-fit** and **best-fit** strategies on a memory map;
- measure **internal** and **external fragmentation** and decide when **compaction** is worthwhile;
- organise memory with **paging**: build page tables and translate logical addresses;
- simulate **demand paging** with the **FIFO** and **LRU** replacement algorithms, count **page faults**, and relate them to what a user actually experiences;
- argue a technical decision in a short written report, as a professional would.

KEY POINTS **Skills assessed**

Analyse a workload, choose and justify a memory management strategy, compute fragmentation and page faults, and communicate a conclusion clearly. These are exactly the competences tested in the GCE Advanced Level problem-solving questions on memory management.

5.2 Situation

Memory Manager of "Molyko Cybercafé".

Molyko Cybercafé, near the University of Buea, has one powerful shared computer used by customers for typing, browsing, photo editing and video editing. The machine has **64 KB of RAM available for user programs**, organised as **16 frames of 4 KB each** (frames numbered 0 to 15). The owner complains: "When many customers open programs, the machine wastes memory; and when the video-editing program runs, the machine keeps freezing." You are the operating system. Your group must diagnose the situation and propose solutions.

Workload for Part A (contiguous allocation). Memory is initially empty. Programs arrive and leave in this order:

Step	Event	Program	Size
1	Arrive	P1 (browser)	12 KB
2	Arrive	P2 (word processor)	8 KB
3	Arrive	P3 (photo editor)	20 KB
4	Arrive	P4 (music player)	6 KB
5	Leave	P2	—
6	Leave	P4	—
7	Arrive	P5 (chat application)	10 KB
8	Arrive	P6 (spreadsheet)	14 KB

Data for Part B (paging). The same machine now uses paging with a **page size of 4 KB**. The video-editing program V has **6 pages** (numbered 0 to 5). At a given moment its page table is:

Page	Frame	Valid bit
0	5	1
1	9	1
2	—	0
3	2	1
4	12	1
5	—	0

Three logical addresses must be translated: $A_1 = 4100$, $A_2 = 14340$, $A_3 = 9000$ (all in bytes, decimal).

Data for Part C (demand paging). The video editor V is so large that only **3 frames** are granted to it. During an editing session, V references its pages in this order (the **reference string**):

0, 1, 2, 3, 0, 1, 4, 0, 1, 2, 3, 4

The three frames are initially empty. Each page fault triggers a disk access of about 8 ms, during which the customer sees the machine "freeze".

5.3 Tasks

Part A — Contiguous allocation.

1. Draw the 64 KB memory map and simulate steps 1 to 8 using **first-fit**. Show the map after each step.
2. Compute the **external fragmentation** (total free memory scattered in holes) after step 6, and again after step 8.
3. Does P6 find space at step 8 with first-fit? Justify from your map.
4. Repeat the simulation with **best-fit**. Is the result different at step 8? Which strategy leaves the largest single hole?

5. Explain what **compaction** would do here, and give one reason why an operating system does not compact memory continuously.
6. Why is there no internal fragmentation in this part?

Part B — Paging.

7. With 64 KB of RAM and 4 KB pages, how many bits are needed for the **offset** within a page? How many pages can a 16-bit logical address space contain?
8. Translate the logical addresses A_1, A_2 and A_3 into physical addresses, or state that a **page fault** occurs. Show every step: page number, offset, frame, physical address.
9. Explain why paging eliminates external fragmentation, and compute the maximum possible internal fragmentation for program V.

Part C — Demand paging.

10. Simulate the reference string with **FIFO** replacement and 3 frames. Use a trace table and count the page faults.
11. Repeat with **LRU**. Count the page faults.
12. Using the 8 ms disk cost, compute the total time lost to page faults under each algorithm. Which algorithm performs better on this workload? Is that always the case?
13. Write your group's one-page memory manager's report: diagnosis of the owner's two complaints (wasted memory, freezing), the strategy you recommend (contiguous with compaction, or paging with demand paging), and a justification using your numerical results.

5.4 Model answer

Part A. Steps 1–4 fill memory contiguously: P1 occupies [0, 12), P2 [12, 20), P3 [20, 40), P4 [40, 46), leaving one hole [46, 64) of 18 KB.

After step 5 (P2 leaves): a hole [12, 20) of 8 KB appears. After step 6 (P4 leaves): a hole [40, 46) of 6 KB appears. External fragmentation after step 6: 8 + 6 + 18 = 32 KB scattered in three holes.

Step 7, P5 (10 KB): first-fit scans from address 0; the holes of 8 KB and 6 KB are too small, so P5 goes into [46, 56), leaving a hole [56, 64) of 8 KB.

Step 8, P6 (14 KB): the holes are now 8 KB, 6 KB and 8 KB. **No single hole is large enough**, although 8 + 6 + 8 = 22 KB is free in total. P6 is blocked by external fragmentation. External fragmentation after step 8 is therefore 22 KB.

With best-fit: at step 7, P5 (10 KB) still only fits in the 18 KB hole (the other holes are smaller than 10 KB), so the maps are identical here and P6 is still blocked. Best-fit would matter if P5 were smaller: e.g. a 6 KB program would be placed exactly in the 6 KB hole, preserving the larger holes. In general neither first-fit nor best-fit can guarantee that a large program will find a large enough single hole.

Compaction would slide P3 and P5 down to close the gaps, creating one hole of 22 KB, enough for P6. It is not done continuously because it requires moving programs in memory, updating all their addresses (relocation), and stopping them during the move — a heavy cost in CPU time.

There is no internal fragmentation because each program receives exactly the number of bytes it requests (variable-size partitions).

Part B. Page size = 4 KB = 4096 = 2^{12} bytes, so the **offset** needs **12 bits**. A 16-bit logical address leaves 16 − 12 = 4 bits for the page number, i.e. 2^4 = 16 pages.

Translation (page = $\lfloor A/4096 \rfloor$, offset = A mod 4096, physical = frame × 4096 + offset):

A_1 = 4100 : page 1, offset 4 → frame 9 → 9 × 4096 + 4 = 36868.
A_2 = 14340 : page 3, offset 2052 → frame 2 → 2 × 4096 + 2052 = 10244.
A_3 = 9000 : page 2, offset 808 → valid bit 0 → **page fault**.

Paging eliminates external fragmentation because any free frame can hold any page: free memory is never "too small a hole". Internal fragmentation concerns only the last page of a program; for V (6 pages), at most one page is partially used, so internal fragmentation is strictly less than one page, i.e. at most 4095 bytes.

Part C. FIFO trace (F = fault; the page replaced is always the one that entered earliest):

Ref	0	1	2	3	0	1	4	0	1	2	3	4
F1	0	0	0	3	3	3	4	4	4	4	4	4
F2	—	1	1	1	0	0	0	0	0	2	2	2
F3	—	—	2	2	2	1	1	1	1	1	3	3
Fault	F	F	F	F	F	F	F			F	F	

FIFO gives **9 page faults** out of 12 references. Note the last reference (page 4) is a hit: page 4 entered at the 7th reference and, being the most recently loaded, it survives the replacements of pages 0 and 1.

LRU trace (the page replaced is the one not used for the longest time):

Ref	0	1	2	3	0	1	4	0	1	2	3	4
F1	0	0	0	3	3	3	4	4	4	2	2	2
F2	—	1	1	1	0	0	0	0	0	0	3	3
F3	—	—	2	2	2	1	1	1	1	1	1	4
Fault	F	F	F	F	F	F	F			F	F	F

LRU gives **10 page faults** on this particular string. Explanation of the difference: at the 10th reference (page 2), LRU evicts page 4 (unused since the 7th reference), whereas FIFO evicts page 0 (the oldest loaded). Page 4 is then needed again at the very last reference: FIFO still has it (hit), LRU must reload it (fault).

Time lost: FIFO 9 × 8 ms = 72 ms; LRU 10 × 8 ms = 80 ms of freezing per editing session. **On this workload FIFO happens to win.** This must not be generalised: on most real workloads, which exhibit strong **locality of reference**, LRU performs at least as well as FIFO, and unlike FIFO it never suffers from **Belady's anomaly** (the paradox in which adding frames increases the number of faults). The deeper lesson is that with only 3 frames for a 6-page program, any algorithm

faults very often: the real cure for the freezing is to grant V more frames (more RAM), which reduces the working-set pressure far more than the choice of algorithm.

The report should therefore recommend: paging with demand paging (this alone removes the 22 KB of unusable holes, replacing them with at most 4 KB of internal fragmentation per program), a replacement algorithm such as LRU that exploits locality and avoids Belady's anomaly, and — decisively for the freezing complaint — adding RAM so that V receives more frames and the page-fault rate drops.

WARNING **Common mistakes to avoid in your report**

Do not confuse external fragmentation (lost between partitions, contiguous allocation) with internal fragmentation (lost inside the last page, paging). When translating an address, the offset is copied unchanged — never add the frame number to the offset. In FIFO, replace the page that entered first, not the one unused for the longest time; in LRU, use the time of last reference, not the time of loading. Finally, never claim that LRU is always better than FIFO: justify every conclusion with the trace table you actually computed.

6 METHODS AND WORKED EXAMPLES

7 MEMORY MANAGEMENT BASICS

DEFINITION **Logical vs Physical Address Space**

The **logical address** (or virtual address) is generated by the CPU during program execution. The **physical address** is the actual address seen by the memory unit. The **Memory Management Unit (MMU)** performs the runtime mapping from logical to physical addresses. The set of all logical addresses generated by a program is its **logical address space**; the set of corresponding physical addresses is the **physical address space**.

DEFINITION **Memory Management Requirements**

Any memory management scheme must satisfy five fundamental requirements:

1. **Relocation**: A process must be able to reside in different parts of physical memory at different times.
2. **Protection**: Processes must not access memory belonging to other processes or the OS without explicit permission.
3. **Sharing**: Controlled sharing of memory regions (e.g., shared libraries, inter-process communication) must be possible.
4. **Logical Organization**: Programs are structured into modules (code, data, stack) that should be managed independently.
5. **Physical Organization**: Memory hierarchy (cache, RAM, disk) must be managed transparently to the programmer.

PROPERTY **Base and Limit Registers — Dynamic Relocation**

In a simple **contiguous allocation** scheme with dynamic relocation, two hardware registers are used:

- **Base Register** (relocation register): holds the starting physical address of the process.
- **Limit Register**: holds the size (length) of the process in bytes.

For a logical address L:

- If $L \geq$ limit → **address error** (trap to OS).
- Else physical address P = base + L.

This provides both relocation (via base) and protection (via limit). The OS loads base and limit during context switch.

METHOD **Computing Physical Addresses with Base and Limit Registers**

1. Identify the **base register** value (starting physical address of the process) and the **limit register** value (size of the process in bytes).
2. For a given **logical address** L (generated by the CPU), check if $L <$ limit. If $L \geq$ limit, raise an **address error** (trap to OS).
3. If valid, the **physical address** P = base + L.
4. Remember: this scheme provides **relocation** (base) and **protection** (limit). It is used in **contiguous memory allocation** with dynamic relocation.

WORKED EXAMPLE 6 **Base/Limit Address Translation**

A process is loaded into memory starting at physical address `0x20000` (131072 decimal) and has a size of `0x8000` bytes (32768 decimal). The base register holds `0x20000` and the limit register holds `0x8000`. For each of the following logical addresses (hexadecimal), determine whether the address is valid and, if so, compute the corresponding physical address (hexadecimal).

1. `0x1000`
2. `0x7FFF`
3. `0x8000`
4. `0xA000`

SOLUTION

Solution:

- Base = `0x20000` = 131072, Limit = `0x8000` = 32768.
- Valid logical addresses are in range [0, limit − 1] = [0, `0x7FFF`].

1. `0x1000` = 4096. Since 4096 < 32768, valid. Physical address = `0x20000` + `0x1000` =

`0x21000`.

2. `0x7FFF` = 32767. Valid (equal to limit−1). Physical address = `0x20000` + `0x7FFF` = `0x27FFF`.
3. `0x8000` = 32768. This equals the limit, so $L \geq$ limit → invalid (address error trap).
4. `0xA000` = 40960. 40960 > 32768 → invalid (address error trap).

Key point: The limit register holds the size, not the last valid address. Always compare L with limit (not limit−1) for the trap condition.

KEY POINTS **Swapping**

When a process is waiting for I/O or is low priority, the OS can **swap** it out to a **backing store** (disk) to free memory for other processes. Later it is swapped back in (possibly at a different physical address → relocation). Swapping requires dynamic relocation (base/limit or paging). The total physical memory needed at any time is the sum of resident processes; swapping allows the sum of logical address spaces to exceed physical memory.

EXPERIMENT **Observing Memory Layout in Linux**

On a Linux terminal (e.g., in a school lab or on OnBuch+ virtual environment), run:

```
cat /proc/self/maps
```

You will see the logical address ranges (start-end) for code, heap, stack, shared libraries, etc., with permissions (rwxp). This illustrates the logical organization of a process.

8 CONTIGUOUS MEMORY ALLOCATION

DEFINITION **Contiguous Memory Allocation**

Each process is loaded into a single **contiguous block** of physical memory. The OS maintains a table of **free partitions** (holes) and **allocated partitions**. Two main schemes:

- **Fixed Partitioning**: Memory divided into fixed-size partitions at boot time. Simple but suffers from **internal fragmentation** (wasted space inside a partition allocated to a smaller process).

- **Dynamic Partitioning**: Partitions created dynamically to fit each process exactly. No internal fragmentation, but suffers from external fragmentation (free memory broken into small non-contiguous holes).

PROPERTY **Fragmentation**

- **Internal Fragmentation**: Allocated memory slightly larger than requested; the unused portion inside the partition is wasted. Occurs in fixed partitioning and paging (last page).
- **External Fragmentation**: Free memory exists but is not contiguous; cannot satisfy a request even though total free ≥ request. Occurs in dynamic partitioning and segmentation.

Compaction (moving processes to gather all free space into one large block) can reduce external fragmentation but requires dynamic relocation and is CPU-intensive.

METHOD **Contiguous Allocation: First-Fit, Best-Fit, Worst-Fit**

1. List the **free memory partitions** (holes) in order of their starting addresses, each with its size.
2. For each process request (in arrival order):
   - First-Fit: Scan holes from the beginning; allocate the first hole large enough. Reduce that hole's size (or remove it if exact fit).
   - Best-Fit: Scan all holes; allocate the smallest hole that is large enough (minimises leftover fragment).
   - Worst-Fit: Scan all holes; allocate the largest hole (aims to leave a large leftover fragment).
3. If no hole is large enough, the process must wait (or trigger compaction).
4. After each allocation, update the hole list (split if necessary). Keep track of **external fragmentation** (total free memory that is not contiguous).

WORKED EXAMPLE 7 **Contiguous Allocation Algorithms**

Memory partitions (holes) available at start (addresses in KB for simplicity):

Hole	Start (KB)	Size (KB)
H1	0	20
H2	50	30
H3	120	40
H4	200	10

Processes arrive in order: P1 (15 KB), P2 (25 KB), P3 (8 KB), P4 (35 KB). Show the allocation for First-Fit, Best-Fit, and Worst-Fit. Indicate which processes cannot be allocated.

SOLUTION

First-Fit:

- P1 (15): first hole ≥15 is H1 (20). Allocate P1 in H1 → H1 becomes 5 KB at start 15 (or split: 5 KB at 15). Remaining holes: 5@15, 30@50, 40@120, 10@200.
- P2 (25): first hole ≥25 is 30@50 (H2). Allocate P2 → H2 becomes 5 KB at 75. Holes: 5@15, 5@75, 40@120, 10@200.
- P3 (8): first hole ≥8 is 40@120 (H3). Allocate P3 → H3 becomes 32 KB at 128. Holes: 5@15, 5@75, 32@128, 10@200.
- P4 (35): first hole ≥35? 5,5,32,10 → none. P4 waits.

Best-Fit:

- P1 (15): smallest hole ≥15 is H1 (20). Allocate → 5@15 left. Holes: 5@15, 30@50, 40@120, 10@200.
- P2 (25): smallest hole ≥25 is H2 (30). Allocate → 5@75 left. Holes: 5@15, 5@75, 40@120, 10@200.
- P3 (8): smallest hole ≥8 is H4 (10). Allocate → 2@200 left. Holes: 5@15, 5@75, 40@120, 2@200.
- P4 (35): smallest hole ≥35 is H3 (40). Allocate → 5@128 left. All processes allocated.

Worst-Fit:

- P1 (15): largest hole is H3 (40). Allocate → 25@120 left. Holes: 20@0, 30@50, 25@120, 10@200.
- P2 (25): largest hole is H2 (30) or H3 (25)? Largest is 30@50. Allocate → 5@75 left. Holes: 20@0, 5@75, 25@120, 10@200.
- P3 (8): largest hole is 25@120. Allocate → 17@128 left. Holes: 20@0, 5@75, 17@128, 10@200.
- P4 (35): largest hole is 20@0 (too small), 17,10,5 → none. P4 waits.

Observation: Best-Fit succeeded for all; First-Fit and Worst-Fit left P4 unallocated. Best-Fit tends to leave many small fragments (external fragmentation).

WARNING **Common Mistakes in Allocation Algorithms**

- Forgetting to update the hole list after each allocation (splitting holes).
- Confusing Best-Fit (smallest adequate hole) with Worst-Fit (largest hole).
- Not checking if a process fits before allocating; always verify size ≤ hole size.
- In First-Fit, scanning from the beginning of the hole list each time, not from the last allocation (that is Next-Fit).

EXERCISE 5 **Contiguous Allocation Practice**

Memory holes (start KB, size KB): (0, 50), (100, 30), (200, 20), (300, 60). Processes arrive: A(40), B(25), C(15), D(35), E(10). Simulate First-Fit, Best-Fit, Worst-Fit. Which algorithm leaves the largest single free block at the end?

9 NON-CONTIGUOUS MEMORY ALLOCATION — PAGING AND SEGMENTATION

DEFINITION **Paging**

Paging divides physical memory into fixed-size blocks called **frames** and logical memory into same-size blocks called **pages**. Page size is a power of 2 (typically 4 KB to 64 KB). A **page table** maps each page number to a frame number. No external fragmentation (any free frame can hold any page), but internal fragmentation in the last page (average half a page wasted per process).

PROPERTY **Logical Address Structure in Paging**

For a page size $P = 2^p$ bytes and logical address space of 2^m bytes:

- **Page number** $p = \lfloor L / P \rfloor$ (high-order $m - p$ bits).
- **Page offset** $d = L \bmod P$ (low-order p bits).

Physical address = frame_number × P + d. The **Page Table Base Register (PTBR)** points to the page table in memory.

METHOD **Paging: Logical to Physical Address Translation**

1. Determine the **page size** P (usually a power of 2). The logical address is split into:
   - **Page number** $p = \lfloor L / P \rfloor$ (or high-order bits).
   - **Page offset** $d = L \bmod P$ (or low-order bits).
2. Use the **page table** of the process: each entry contains the **frame number** f if the page is in memory, plus a **valid/invalid bit**.
3. If valid bit = 0 → **page fault** (trap to OS).
4. If valid bit = 1, physical address = $f \times P + d$.
5. The page table base register (PTBR) points to the page table in memory.

WORKED EXAMPLE 8 **Paging Address Translation**

A system uses a page size of `4 KB` (2^{12} bytes). A process has the following page table (only valid entries shown):

Page Number	Frame Number	Valid
0	5	1
1	2	1
2	7	1
3	0	0

Translate the following logical addresses (hexadecimal) to physical addresses (hexadecimal) or indicate a page fault.

1. `0x0000`
2. `0x1FFC`
3. `0x2000`
4. `0x3FFC`
5. `0x4000`

SOLUTION

Page size = 4 KB = `0x1000` bytes. Offset = low 12 bits (3 hex digits). Page number = remaining high bits.

1. `0x0000`: page = 0, offset = `0x000`. Page 0 valid, frame 5. Physical = 5 × `0x1000` + `0x000` = `0x5000`.

2. `0x1FFC`: `0x1FFC` = 8188 decimal. 8188/4096 = 1 remainder 4092. So page = 1, offset = `0xFFC`. Page 1 valid, frame 2. Physical = 2 × `0x1000` + `0xFFC` = `0x2FFC`.
3. `0x2000`: `0x2000` = 8192. 8192/4096 = 2 exactly, so page = 2, offset = 0. Page 2 valid, frame 7. Physical = 7 × `0x1000` + 0 = `0x7000`.
4. `0x3FFC`: `0x3FFC` = 16380. 16380/4096 = 3 remainder 4092. Page = 3, offset = `0xFFC`. Page 3 valid bit = 0 → Page Fault.
5. `0x4000`: page = 4 (not in table). Assuming invalid → Page Fault (or trap if page table length exceeded).

Note: Always convert hex to decimal or use bitwise reasoning: page number = address » 12, offset = address & 0×FFF.

DEFINITION **Translation Lookaside Buffer (TLB)**

The **TLB** is a hardware cache for page table entries. It stores recently used (page, frame) pairs. On a memory reference:

- **TLB hit**: frame obtained directly → one memory access for data.
- **TLB miss**: page table consulted in memory → two memory accesses (page table + data), then TLB updated.

TLB hit ratio h significantly reduces effective access time. TLB entries include ASID (Address Space Identifier) for multi-process support.

PROPERTY **Effective Access Time with TLB**

Let t_t = TLB access time, t_m = memory access time, h = TLB hit ratio, p = page fault rate, t_p = page fault service time.

$$\text{EAT} = h(t_t + t_m) + (1 - h)(t_t + 2t_m) + p \cdot t_p$$

If $t_t \ll t_m$ and p is small, often approximated as $\text{EAT} \approx t_m + (1 - h)t_m + pt_p$.

DEFINITION **Segmentation**

Segmentation divides logical memory into variable-sized **segments** corresponding to logical units (code, data, stack, heap). A logical address is a pair $\langle s, d \rangle$ where s = segment number, d = offset. A **segment table** maps each segment to a **base** (physical start) and **limit** (length). Supports sharing (same segment in multiple tables) and protection (r/w/x

per segment). Suffers from external fragmentation.

METHOD **Segmentation: Logical to Physical Address Translation**

1. A logical address in segmentation is a pair $\langle s, d \rangle$ where s = **segment number**, d = **offset within segment**.
2. The **segment table** maps each segment number to a **base** (physical start address) and a **limit** (segment length).
3. For a given $\langle s, d \rangle$:
   (a) Check if s is within the segment table length (STLR). If not → trap.
   (b) Check if d < limit[s]. If not → **segmentation fault** (trap).
   (c) Physical address = base[s] + d.
4. Segmentation supports **sharing** (same base/limit in multiple tables) and **protection** (read/write/execute bits per segment).

WORKED EXAMPLE 9 **Segmentation Address Translation**

A process has the following segment table:

Segment	Base (hex)	Limit (hex)	Protection
0	0x10000	0x2000	RW
1	0x30000	0x1000	RX
2	0x50000	0x4000	RW

For each logical address $\langle s, d \rangle$ (hex), determine the physical address or the type of fault.

1. $\langle 0, \texttt{0x1500} \rangle$
2. $\langle 1, \texttt{0x1000} \rangle$
3. $\langle 2, \texttt{0x3FFF} \rangle$
4. $\langle 3, \texttt{0x0100} \rangle$
5. $\langle 1, \texttt{0x0800} \rangle$ (attempt to write)

SOLUTION

1. Segment 0: base=0x10000, limit=0x2000=8192. Offset 0x1500=5376 < 8192 → valid.

Physical = 0x10000 + 0x1500 = 0x11500. Protection RW allows read/write.

2. Segment 1: base=0x30000, limit=0x1000=4096. Offset 0x1000=4096. Since offset must be strictly less than limit, 4096 ≮ 4096 → Segmentation Fault (limit violation).
3. Segment 2: base=0x50000, limit=0x4000=16384. Offset 0x3FFF=16383 < 16384 → valid. Physical = 0x50000 + 0x3FFF = 0x53FFF. Protection RW.
4. Segment 3: not in table (only segments 0–2). → Invalid Segment Fault (trap to OS).
5. Segment 1, offset 0x0800=2048 < 4096 → valid address. Physical = 0x30000 + 0x0800 = 0x30800. But protection is RX (read/execute). Write attempt → Protection Fault.

KEY POINTS **Paging vs Segmentation Comparison**

Aspect	Paging	Segmentation
Unit size	Fixed (page/frame)	Variable (segment)
Fragmentation	Internal only	External
Address visible to programmer	No (single linear space)	Yes ($\langle s, d \rangle$)
Sharing	Page-level (less natural)	Segment-level (natural: code, data)
Protection	Page-level (r/w/x)	Segment-level (r/w/x)
Table size	Large (one entry per page)	Small (one entry per segment)

DID YOU KNOW? **Intel x86 Architecture**

The Intel x86 architecture (used in most PCs in Cameroon and worldwide) historically combined segmentation and paging. In 32-bit protected mode: logical address → segmentation unit → linear address → paging unit → physical address. Modern 64-bit mode uses a flat segmentation model (effectively disabling segmentation) and relies on 4-level paging.

10 VIRTUAL MEMORY AND DEMAND PAGING

DEFINITION **Virtual Memory**

Virtual Memory allows execution of processes that are not completely in physical memory. Only the needed pages (or segments) are loaded **on demand**. Benefits:

- Programs larger than physical memory can run.
- More processes can reside in memory → higher CPU utilization.
- Less I/O for loading/swapping (only needed pages).

Implemented via **demand paging** (pages) or **demand segmentation**.

PROPERTY **Page Fault Handling**

When a process references a page with valid bit = 0:

1. Trap to OS (page fault interrupt).
2. OS checks internal table: invalid reference → terminate process; valid but not in memory → proceed.
3. Find a free frame (if none, run **page replacement** algorithm to select a victim).
4. Schedule disk read to bring page into frame.
5. Update page table (frame number, valid=1).
6. Restart the faulting instruction.

METHOD **Page Replacement Algorithms: FIFO, LRU, Optimal**

1. Given a **reference string** (sequence of page numbers) and a number of **frames** N.
2. Simulate each algorithm by maintaining an ordered list of pages currently in frames.
3. FIFO: Replace the page that has been in memory the longest (queue). On page fault, if frames not full, add to tail; else remove head, add new to tail.
4. LRU: Replace the page least recently used (stack or timestamp). On reference, move page to most-recent position. On fault, evict least-recent.
5. Optimal (OPT): Replace the page that will not be used for the longest time in the future (requires future knowledge). Used as benchmark.
6. Count **page faults** for each algorithm. Fewer faults = better.
7. Belady's Anomaly: FIFO may exhibit more faults with more frames; LRU and OPT do

not (they satisfy the stack property).

WORKED EXAMPLE 10 **Page Replacement Simulation**

Reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5. Number of frames = 3. Initially empty. Compute the number of page faults for FIFO, LRU, and Optimal.

SOLUTION

We trace each algorithm. Faults marked with *.
FIFO (queue):

Ref	Frame1	Frame2	Frame3	Fault?
1	1*	-	-	*
2	1	2*	-	*
3	1	2	3*	*
4	4*	2	3	* (evict 1)
1	4	1*	3	* (evict 2)
2	4	1	2*	* (evict 3)
5	5*	1	2	* (evict 4)
1	5	1	2	hit
2	5	1	2	hit
3	3*	1	2	* (evict 5)
4	3	4*	2	* (evict 1)
5	3	4	5*	* (evict 2)

Total FIFO faults = 9.
LRU (stack, most recent at top):

Ref	Frame1	Frame2	Frame3	Fault?
1	1*	-	-	*
2	2*	1	-	*
3	3*	2	1	*
4	4*	3	2	* (evict 1)
1	1*	4	3	* (evict 2)
2	2*	1	4	* (evict 3)
5	5*	2	1	* (evict 4)
1	1	5	2	hit (move 1 to top)
2	2	1	5	hit
3	3*	2	1	* (evict 5)
4	4*	3	2	* (evict 1)
5	5*	4	3	* (evict 2)

Total LRU faults = 10.
Optimal (look ahead):

Ref	Frame1	Frame2	Frame3	
1	1*	-	-	
2	1	2*	-	
3	1	2	3*	
4	4*	2	3	* (evict 1? Future: 1 at po
1	1	2	4	
2	1	2	4	
5	5*	2	4	* (evict 1? Future: 1 none, 2 at 10? Actually after pos7 (5), refs: 1,2,3
1	1	2	5	
2	1	2	5	
3	3*	2	5	* (evict
4	4*	2	5	* (evict 2? Future: 5 at 12
5	3	4	5	

Total Optimal faults = 7.
Summary: FIFO=9, LRU=10, Optimal=7. Note: LRU worse than FIFO here (no stack property violation, just particular string). Optimal is lower bound.

WARNING **Belady's Anomaly**

FIFO can suffer from Belady's Anomaly: increasing the number of frames may increase page faults. Example: reference string 1,2,3,4,1,2,5,1,2,3,4,5 with 3 frames gives 9 faults, with 4 frames gives 10 faults. LRU and Optimal never exhibit this anomaly (they are **stack algorithms**).

METHOD **Additional Page Replacement Algorithms**

1. Second Chance (Clock): FIFO with a reference bit. On fault, inspect head: if ref bit = 0, replace; if ref bit = 1, clear bit, move to tail, repeat. Approximates LRU.
2. Enhanced Second Chance: Uses (reference, modify) bits: prefer (0,0) > (0,1) > (1,0) > (1,1) to avoid writing dirty pages.
3. LFU (Least Frequently Used): Replace page with smallest access count. May keep pages used heavily in past but not now.
4. MFU (Most Frequently Used): Replace page with largest count (assumes it won't be used again soon). Rarely used.

PROPERTY **Frame Allocation Policies**

How to distribute a fixed number of free frames among processes:

- **Equal Allocation**: Each process gets $\lfloor \text{frames}/\text{processes} \rfloor$.
- **Proportional Allocation**: Frames proportional to process size (or priority).
- **Priority Allocation**: Higher priority processes get more frames; can take frames from lower priority.
- **Global vs Local Replacement**: Global = victim chosen from all frames; Local = victim chosen only from frames of faulting process. Local prevents one process's thrashing from affecting others.

METHOD **Effective Access Time (EAT) with Page Faults**

1. Identify:
   - t_m = memory access time (for a page table lookup + data access, or just data if TLB hit).

- t_p = page fault service time (disk I/O + OS overhead + restart).
- p = page fault rate (probability of a page fault per memory access).

2. The **Effective Access Time** (average) is:

$$\text{EAT} = (1 - p) \times t_m + p \times (t_p + t_m)$$

(assuming on a page fault we still need the memory access after service). Often simplified to EAT = $t_m + p \times t_p$ if $t_p \gg t_m$.

3. If a **TLB** (Translation Lookaside Buffer) is present with hit ratio h, TLB access time t_t, then:

$$\text{EAT} = h \times (t_t + t_m) + (1 - h) \times (t_t + 2t_m) + p \times t_p$$

(assuming page table in memory, two memory accesses on TLB miss). Adjust formula as per system description.

WORKED EXAMPLE 11 **Effective Access Time Calculation**

A demand-paged system has the following parameters:

- Memory access time t_m = 100 ns.
- Page fault service time t_p = 8 ms = 8 000 000 ns.
- Page fault rate p = 0.0001 (0.01%).

Compute the Effective Access Time (EAT) in nanoseconds and in milliseconds. What is the performance degradation factor compared to a system with no page faults?

SOLUTION

Using EAT = $t_m + p \times t_p$ (since $t_p \gg t_m$):

$$\text{EAT} = 100 + 0.0001 \times 8\,000\,000 = 100 + 800 = 900 \text{ ns}.$$

In milliseconds: 900 ns = 0.0009 ms. Without page faults, access time = t_m = 100 ns. Degradation factor = $\frac{900}{100}$ = 9. The system runs 9 times slower on average due to page faults, even though the fault rate is only 0.01%. Lesson: Because disk I/O is orders of magnitude slower than memory, even a tiny page fault rate drastically increases EAT. This motivates keeping the page fault rate very low (good replacement algorithms, sufficient frames, working set model).

DEFINITION **Thrashing**

Thrashing occurs when a process spends more time paging (handling page faults) than executing. Cause: total memory demand (sum of working sets) exceeds physical frames → constant page faults → CPU utilization drops → OS increases multiprogramming → worse thrashing.

METHOD **Working Set Model and Page Fault Frequency**

1. The **working set** $W(t, \Delta)$ of a process at time t with window Δ is the set of pages referenced in the last Δ time units (or Δ memory references).
2. Working Set Strategy: OS monitors each process's working set size. Allocate at least $|W|$ frames to the process. If total demand > available frames → **thrashing** (suspend some processes).
3. Page Fault Frequency (PFF): Set an acceptable page fault rate interval $[r_{min}, r_{max}]$. Monitor actual fault rate.
   - If rate > r_{max} → increase frames allocated.
   - If rate < r_{min} → decrease frames (give to others).
4. Both aim to keep the process's **locality** in memory, minimising faults.

WORKED EXAMPLE 12 **Working Set and Thrashing**

A system has 100 free frames. Three processes have current working set sizes (estimated over window Δ): P1: 30 pages, P2: 40 pages, P3: 50 pages.

1. Can all three run without thrashing? If not, which should be suspended?
2. Suppose P1's page fault rate is monitored. Acceptable range: 0.5% to 2%. Current rate: 3%. What action should the OS take?
3. If P2's fault rate is 0.2%, what action?

SOLUTION

1. Total working set demand = 30+40+50 = 120 pages > 100 frames. Thrashing will occur if all three run. The OS should suspend one process (typically the largest or lowest priority) to reduce demand. Suspending P3 (50) leaves 70 demand < 100, comfortable. Or suspend P2 (40) leaves 80. Decision depends on priority.
2. P1 fault rate 3% > upper bound 2% → increase frames allocated to P1 (if free frames exist, or take from a process with low fault rate).

3. P2 fault rate 0.2% < lower bound 0.5% → decrease frames allocated to P2 (it has more frames than needed for its locality). Reclaim frames for other processes.

Key idea: Working set and PFF are dynamic strategies to prevent thrashing by matching allocated frames to the process's current locality needs.

METHOD **Integrated Memory Management: Paging + Segmentation**

1. In a combined **segmented paging** system, the logical address is $\langle s, p, d \rangle$:
   - s = segment number → segment table entry gives base address of **page table** for that segment.
   - p = page number within segment → index into that page table to get frame number f.
   - d = page offset.
2. Physical address = $f \times \text{page_size} + d$.
3. Protection and sharing are at segment level; allocation is in page frames (no external fragmentation).
4. Steps for translation:
   (a) Use s to index segment table; check segment limit (if p exceeds segment pages → fault).
   (b) Get page table base from segment entry.
   (c) Use p to index page table; check valid bit.
   (d) Combine frame f with offset d.

WORKED EXAMPLE 13 **Segmented Paging Address Translation**

A system uses segmented paging with page size = `1 KB` (2^{10} bytes). Segment table for a process:

Segment	Page Table Base (physical)	Segment Limit (pages)
0	0x2000	4
1	0x3000	8
2	0x4000	2

Page tables (each entry 2 bytes: frame number in high 12 bits, valid bit in bit 0):

- Page Table 0 (at `0x2000`): entries for pages 0–3: [`0x5001`, `0x6001`, `0x7001`, `0x0000`] (last invalid).
- Page Table 1 (at `0x3000`): entries for pages 0–7: all valid, frames 10,11,12,13,14,15,16,17 (i.e., `0xA001`, `0xB001`, ...).
- Page Table 2 (at `0x4000`): entries for pages 0–1: [`0x8001`, `0x9001`].

Translate logical address $\langle s = 1, p = 3, d = 0x200 \rangle$ (hex). Show all steps.

SOLUTION

Step 1: Segment $s = 1$. Segment table entry: page table base = `0x3000`, limit = 8 pages. Page $p = 3 < 8 \rightarrow$ valid segment offset. Step 2: Page table for segment 1 starts at `0x3000`. Each entry 2 bytes. Entry for page 3 is at `0x3000` + 3 × 2 = `0x3006`. Step 3: Read entry at `0x3006`. Given frames: page 3 → frame 13 = `0xD`. Entry = `0xD001` (frame in high 12 bits, valid=1). Valid bit = 1 → page in memory. Frame number f = `0xD` (13 decimal). Step 4: Page size = `1 KB` = `0x400` bytes. Offset d = `0x200`. Physical address = f × page_size + d = `0xD` × `0x400` + `0x200` = `0x3400` + `0x200` = `0x3600`. Result: Physical address = `0x3600`.

DID YOU KNOW? **Memory Management in Modern OS**

Modern OS (Linux, Windows, macOS) use **demand paging** with **copy-on-write** (fork shares pages, copies only on write), **memory-mapped files** (file I/O via page faults), and **swap space** on SSD/HDD. The **Clock-Pro** and **ARC** algorithms improve on LRU by tracking frequency and recency. In Cameroon's growing tech ecosystem (e.g., Silicon Mountain in Buea), understanding virtual memory is crucial for optimizing server applications and mobile apps.

EXERCISE 6 **Virtual Memory Comprehensive**

1. A system has 32-bit logical addresses, 4 KB pages, and 512 MB physical memory. How many entries in a single-level page table? How many bits for frame number?
2. Reference string: 7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1. Frames = 3. Simulate FIFO, LRU, Optimal. Which performs best?
3. A process has 4 segments: code (16 KB), data (8 KB), stack (4 KB), heap (12 KB). Page size = 4 KB. How many pages total? If using segmented paging, how many page tables?
4. EAT calculation: t_m = 50 ns, t_p = 10 ms, p = 0.0002. TLB hit ratio h = 0.95, t_t = 10 ns. Compute EAT with and without TLB.

SOLUTION **Exercise 1**

1. Logical address space = 2^{32} bytes. Page size = 2^{12} bytes. Number of pages = 2^{20} = 1 048 576 entries. Physical memory = 512 MB = 2^{29} bytes. Number of frames = $2^{29}/2^{12}$ = 2^{17}. Frame number needs 17 bits. Page table entry size ≥ 17 bits + control bits.
2. (Student to simulate) Optimal typically best.
3. Code: 4 pages, Data: 2 pages, Stack: 1 page, Heap: 3 pages. Total = 10 pages. 4 page tables (one per segment).
4. Without TLB: EAT = $50 + 0.0002 \times 10^7 = 50 + 2000 = 2050$ ns. With TLB: EAT = $0.95(10 + 50) + 0.05(10 + 100) + 0.0002 \times 10^7 = 57 + 5.5 + 2000 = 2062.5$ ns. (TLB helps only on TLB hits; page fault dominates).

EXPERIMENT **Simulating Page Replacement in Python**

Write a short Python script to simulate FIFO, LRU, Optimal for a given reference string and frame count. This reinforces understanding and prepares for practical exam questions.

```
def fifo(refs, frames):
    mem, faults = [], 0
    for r in refs:
        if r not in mem:
            faults += 1
            if len(mem) < frames:
                mem.append(r)
            else:
                mem.pop(0)
                mem.append(r)
    return faults
# Similar for LRU (use list, move to end on hit) and Optimal (look ahead).
```

11 EXERCISES

11.1 I check my knowledge

EXERCISE 7 **Multiple choice questions**

For each question, choose the single correct answer.

1. The register that holds the starting physical address of a process in memory is the:
   (a) limit register
   (b) base (relocation) register
   (c) page table register
   (d) program counter
2. External fragmentation is a problem associated mainly with:
   (a) paging
   (b) contiguous allocation with variable-size partitions
   (c) demand paging
   (d) segmentation with fixed-size segments
3. In a paging system with page size 4 KB, the offset field of a logical address contains:
   (a) 4 bits
   (b) 10 bits
   (c) 12 bits
   (d) 16 bits
4. Thrashing occurs when:
   (a) the CPU is idle because all processes are blocked on input/output
   (b) the system spends more time swapping pages than executing processes
   (c) a page fault is caused by an invalid address
   (d) two processes share the same frame

EXERCISE 8 **True or false — justify**

State whether each statement is true or false, and justify your answer in one or two sentences.

1. In paging, a process can suffer from external fragmentation.
2. Best-fit always produces better memory utilisation than first-fit.
3. Increasing the number of page frames can never increase the number of page faults.
4. In demand paging, a page is loaded into main memory only when it is actually referenced.

EXERCISE 9 **Definitions**

Define each of the following terms in one or two clear sentences:

1. logical address and physical address;
2. internal fragmentation;
3. page fault;
4. working set (locality of reference).

EXERCISE 10 **Direct calculations**

A paging system uses a page size of 2 KB (2048 bytes).

1. How many pages does a process of 10 000 bytes occupy? How many bytes are lost to internal fragmentation?
2. A logical address is 16 bits long. How many pages can a process contain at most?
3. Give the page number and the offset of the logical address 5000.

11.2 I practise

EXERCISE 11 **Partition placement strategies**

Given memory holes of 100 KB, 500 KB, 200 KB and 300 KB (in that order in memory), show how the following processes are placed using first-fit, best-fit and worst-fit: P_1 = 212 KB, P_2 = 417 KB, P_3 = 112 KB, P_4 = 426 KB (processes are placed in this order).

1. For each strategy, draw a table showing, for every process, the hole chosen and the

hole remaining after placement.

2. State which processes, if any, cannot be placed under each strategy.
3. Conclude: which strategy makes the best use of memory in this example? Justify.

EXERCISE 12 **Address translation in paging**

A system has a page size of 4 KB (4096 bytes) and the following page table: page 0 → frame 5, page 1 → frame 2, page 2 → frame 9, page 3 → frame 1.

1. Translate the logical addresses 0, 4100, 8300 and 12300 into physical addresses, showing the page number and offset in each case.
2. What happens if the process references the logical address 17000? Explain.

EXERCISE 13 **Fragmentation**

1. Explain clearly the difference between internal and external fragmentation, using a small diagram or numerical example for each.
2. For each of the following techniques — fixed contiguous partitions, variable contiguous partitions, paging, segmentation — state which type of fragmentation it suffers from, and why.
3. Name the technique used to fight external fragmentation in contiguous allocation and explain its main drawback.

EXERCISE 14 **FIFO and LRU simulation**

Consider the reference string 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 with 3 page frames, initially empty.

1. Simulate the FIFO page replacement algorithm using a trace table (one column per reference, one row per frame). Count the page faults.
2. Repeat the simulation with LRU and count the page faults.
3. Compare the two results and explain, in two or three sentences, why one algorithm performs better on this string.

11.3 I prepare for the GCE

EXERCISE 15 **Belady's anomaly — structured question**

A demand-paged system executes a process that references pages in the order

$$1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5.$$

1. Define the terms page fault and page replacement. (2 marks)
2. Simulate FIFO with 3 frames and give the number of page faults. (4 marks)
3. Simulate FIFO with 4 frames and give the number of page faults. (4 marks)
4. What surprising observation do you make? What is the name of this phenomenon? (2 marks)
5. Name one page replacement algorithm that never exhibits this phenomenon, and justify briefly. (2 marks)

EXERCISE 16 **Handling a page fault and thrashing — essay question**

1. Describe, in the correct order, the six steps taken by the operating system when a page fault occurs, from the detection of the invalid reference to the restart of the interrupted instruction. (6 marks)
2. A student in Yaoundé notices that his computer "freezes" whenever many applications are opened at the same time. Using the concepts of thrashing and locality of reference, explain this phenomenon. (4 marks)
3. Propose two remedies the operating system (or the user) can apply to reduce thrashing, and justify each one. (4 marks)

EXERCISE 17 **Address translation by the MMU — structured question**

1. Define logical address and physical address, and explain why the distinction between them is useful. (3 marks)
2. In a system using contiguous allocation with relocation, explain the role of the base register and the limit register during the execution of a process. (4 marks)
3. Draw a clearly labelled diagram showing how the Memory Management Unit (MMU) translates a logical address into a physical address using the base and limit registers,

including the case of an illegal address. (4 marks)

4. A process is loaded at physical address 14000 and has a size of 5000 bytes. For each of the logical addresses 0, 2500, 4999 and 5000, give the physical address produced or state what happens. (4 marks)

12 SOLUTIONS TO THE EXERCISES

SOLUTION **Exercise 1 — Multiple choice questions**

1. Answer: (b) base (relocation) register. The base register (also called relocation register) holds the starting physical address of a process in memory. The limit register holds the size (or the upper bound) of the process. The page table register points to the page table, and the program counter holds the address of the next instruction to execute.

2. Answer: (b) contiguous allocation with variable-size partitions. External fragmentation occurs when free memory is broken into small, non-contiguous blocks that cannot satisfy a request even though their total size is sufficient. This is typical of variable-size partitions (dynamic partitioning). Paging and segmentation with fixed-size segments avoid external fragmentation; demand paging is a virtual memory technique, not an allocation scheme.

3. Answer: (c) 12 bits. Page size = 4 KB = 4096 bytes = 2^{12} bytes. The offset field must be able to address every byte within a page, so it needs $\log_2(4096) = 12$ bits.

4. Answer: (b) the system spends more time swapping pages than executing processes. Thrashing is a condition where the CPU spends most of its time handling page faults (swapping pages in/out) rather than executing useful work, usually because the working sets of active processes exceed the available physical frames.

SOLUTION **Exercise 2 — True or false — justify**

1. False. In paging, memory is divided into fixed-size frames. A process is allocated a set of frames that need not be contiguous. Therefore, there is no external fragmentation (free frames can be used for any page). However, internal fragmentation can occur in the last page of a process.

2. False. Best-fit searches for the smallest hole that fits the request, which often leaves behind many tiny, unusable holes (increasing external fragmentation). First-fit is faster and sometimes leaves larger holes. Neither is universally better; performance depends on the sequence of requests.

3. False. This statement describes Belady's anomaly: for some page reference strings, increasing the number of frames can increase the number of page faults when using FIFO (or other non-stack algorithms). Stack algorithms like LRU never exhibit this anomaly.

4. True. Demand paging loads a page into main memory only when it is referenced (i.e., when a page fault occurs). Pages not referenced are never loaded, saving memory and I/O.

SOLUTION **Exercise 3 — Definitions**

1. Logical address and physical address: A logical address (or virtual address) is the address generated by the CPU during program execution; it is the address the process sees. A physical address is the actual address in main memory (RAM) where the data resides. The Memory Management Unit (MMU) translates logical addresses to physical addresses at runtime.
2. Internal fragmentation: Wasted space within an allocated memory block because the allocated unit (e.g., a page or a fixed partition) is larger than the requested memory. For example, if a process requests 3 KB but is given a 4 KB page, 1 KB is internally fragmented.
3. Page fault: An interrupt (trap) that occurs when a process references a page that is not currently present in main memory (i.e., the valid bit in the page table entry is 0). The OS must then load the required page from secondary storage into a free frame, update the page table, and restart the instruction.
4. Working set (locality of reference): The working set of a process at a given time is the set of pages that have been referenced in the most recent Δ memory references (or within a time window). It models the locality of reference principle: processes tend to access a relatively small, stable set of pages during any phase of execution.

SOLUTION **Exercise 4 — Direct calculations**

Page size = 2 KB = 2048 bytes.

1. Process size = 10 000 bytes. Number of pages = $\lceil 10000/2048 \rceil = \lceil 4.8828 \rceil = 5$ pages. Total allocated space = $5 \times 2048 = 10240$ bytes. Internal fragmentation = $10240 - 10000 = 240$ bytes.
2. Logical address = 16 bits $\Rightarrow$ logical address space = $2^{16} = 65536$ bytes. Maximum number of pages = $\frac{65536}{2048} = 32$ pages. (Alternatively, offset uses $\log_2(2048) = 11$ bits, leaving $16 - 11 = 5$ bits for page number $\Rightarrow 2^5 = 32$ pages.)
3. Logical address 5000 (decimal). Page number = $\lfloor 5000/2048 \rfloor = \lfloor 2.44 \rfloor = 2$. Offset = $5000 \bmod 2048 = 5000 - 2 \times 2048 = 904$. So page number = 2, offset = 904.

SOLUTION **Exercise 5 — Partition placement strategies**

Initial holes (in order): 100 KB, 500 KB, 200 KB, 300 KB. Processes (in order): P_1 = 212 KB, P_2 = 417 KB, P_3 = 112 KB, P_4 = 426 KB.
We simulate each strategy.

First-fit: Allocate the first hole that is large enough.

Process	Size (KB)	Hole chosen (KB)	Hole remaining (KB)	New hole list (KB)
P_1	212	500 (2nd hole)	288	100, 288, 200, 300
P_2	417	300 (4th hole)	too small	—
P_3	112	288 (2nd hole)	176	100, 176, 200, 300
P_4	426	300 (4th hole)	too small	—

P_2 and P_4 cannot be placed.

Best-fit: Allocate the smallest hole that is large enough.

Process	Size (KB)	Hole chosen (KB)	Hole remaining (KB)	New hole list (KB)
P_1	212	300 (4th hole)	88	100, 500, 200, 88
P_2	417	500 (2nd hole)	83	100, 83, 200, 88
P_3	112	200 (3rd hole)	88	100, 83, 88, 88
P_4	426	none (largest hole is 100)	—	—

P_4 cannot be placed.

Worst-fit: Allocate the largest hole.

Process	Size (KB)	Hole chosen (KB)	Hole remaining (KB)	New hole list (KB)
P_1	212	500 (2nd hole)	288	100, 288, 200, 300
P_2	417	300 (4th hole)	too small	—
P_3	112	288 (2nd hole)	176	100, 176, 200, 300
P_4	426	300 (4th hole)	too small	—

P_2 and P_4 cannot be placed.

Conclusion: In this example, best-fit places three processes (P_1, P_2, P_3) while first-fit and worst-fit place only two (P_1, P_3). Best-fit makes the best use of memory here because it leaves the large 500 KB hole for P_2 (417 KB) by using the 300 KB hole for P_1 (212 KB). However, best-fit can suffer from creating many small fragments in other scenarios.

SOLUTION **Exercise 6 — Address translation in paging**

Page size = 4 KB = 4096 bytes. Page table:

Page	Frame
0	5
1	2
2	9
3	1

Offset bits = 12 (since $2^{12} = 4096$). Page number = logical address $\gg 12$ (integer division by 4096). Offset = logical address mod 4096.

1. Logical address 0: page = 0, offset = 0. Frame = 5. Physical address = $5 \times 4096 + 0 = 20480$.
2. Logical address 4100: page = $\lfloor 4100/4096 \rfloor = 1$, offset = 4100 mod 4096 = 4. Frame = 2. Physical address = $2 \times 4096 + 4 = 8196$.
3. Logical address 8300: page = $\lfloor 8300/4096 \rfloor = 2$, offset = 8300 mod 4096 = 108. Frame = 9. Physical address = $9 \times 4096 + 108 = 36972$.
4. Logical address 12300: page = $\lfloor 12300/4096 \rfloor = 3$, offset = 12300 mod 4096 = 12. Frame = 1. Physical address = $1 \times 4096 + 12 = 4108$.
5. Logical address 17000: page = $\lfloor 17000/4096 \rfloor = 4$, offset = 17000 mod 4096 = 616. The page table has entries only for pages 0–3. Page 4 is not in the table (invalid). The MMU will raise a page fault (or a segmentation fault if the address is beyond the process's logical address space). The OS must handle the fault: if the address is valid (within the process's allocated pages), it loads page 4 from disk; otherwise, it terminates the process.

SOLUTION **Exercise 7 — Fragmentation**

1. Internal fragmentation occurs when allocated memory is larger than requested memory; the unused space inside the allocated block is wasted. Example (paging): Page size = 4 KB. Process requests 10 KB → gets 3 pages (12 KB). Last page has 2 KB unused → internal fragmentation = 2 KB.

   External fragmentation occurs when free memory is split into many small non-contiguous blocks, so that a request cannot be satisfied even though total free memory is sufficient. Example (variable partitions): Memory: 100 KB used, 50 KB free, 200 KB used, 60 KB free. Total free = 110 KB. A request for 100 KB fails because no single contiguous hole of 100 KB exists.

2.

Technique	Fragmentation type	Rea
Fixed contiguous partitions	Internal	Partitions are fixed sizes; a process smalle
Variable contiguous partitions	External	Holes of varying sizes appear between alloc
Paging	Internal	Last page of a process may be part
Segmentation	External	Segments are variable-sized; free memo

3. Technique: Compaction (memory shuffling). The OS moves all allocated blocks together to create one large contiguous free block. Main drawback: Compaction is expensive — it requires relocating processes (updating all their base registers and address references) and is only possible if relocation is dynamic (done at execution time). It also causes CPU overhead and may require stopping processes.

SOLUTION **Exercise 8 — FIFO and LRU simulation**

Reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5. Frames = 3, initially empty.

FIFO simulation: We maintain a queue of frames (oldest at front). Page faults marked with *.

Ref	1	2	3	4	1	2	5	1	2	3	4	5
Frame 1	1*	1	1	4*	4	4	5*	5	5	3*	3	3
Frame 2		2*	2	2	1*	1	1	1	2*	2	4*	4
Frame 3			3*	3	3	2*	2	2	2	2	2	5*
Fault?	*	*	*	*	*	*	*			*	*	*

Total page faults = 9.

LRU simulation: We keep track of most recent use; replace the least recently used.

Ref	1	2	3	4	1	2	5	1	2	3	4	5
Frame 1	1*	1	1	4*	4	4	5*	5	5	3*	3	3
Frame 2		2*	2	2	1*	1	1	1	2*	2	4*	4
Frame 3			3*	3	3	2*	2	2	2	2	2	5*
Fault?	*	*	*	*	*	*	*			*	*	*

Total page faults = 10.

Detailed LRU trace:

- Ref 1: fault, frames [1] (MRU: 1)
- Ref 2: fault, frames [1,2] (MRU: 2,1)
- Ref 3: fault, frames [1,2,3] (MRU: 3,2,1)
- Ref 4: fault, replace LRU (1), frames [4,2,3] (MRU: 4,3,2)
- Ref 1: fault, replace LRU (2), frames [4,1,3] (MRU: 1,4,3)
- Ref 2: fault, replace LRU (3), frames [4,1,2] (MRU: 2,1,4)
- Ref 5: fault, replace LRU (4), frames [5,1,2] (MRU: 5,2,1)
- Ref 1: hit, update MRU: (1,5,2)
- Ref 2: hit, update MRU: (2,1,5)
- Ref 3: fault, replace LRU (5), frames [3,1,2] (MRU: 3,2,1)
- Ref 4: fault, replace LRU (1), frames [3,4,2] (MRU: 4,3,2)
- Ref 5: fault, replace LRU (2), frames [3,4,5] (MRU: 5,4,3)

Comparison: On this reference string, FIFO (9 faults) outperforms LRU (10 faults). This is a known case where LRU's assumption of temporal locality fails because the reference string has a looping pattern that defeats LRU's stack property. FIFO's rigid queue order happens to keep pages 1 and 2 longer in this specific sequence. In general, LRU approximates optimal replacement better than FIFO, but neither dominates the other for all strings.

SOLUTION **Exercise 9 — Belady's anomaly — structured question**

1. Page fault: An interrupt that occurs when a process references a page not currently in main memory (valid bit = 0). The OS must load the page from disk. (1 mark) Page replacement: When a page fault occurs and no free frame exists, the OS selects a victim page in memory, writes it back to disk if dirty, and replaces it with the required page. (1 mark)
2. FIFO with 3 frames: (Same simulation as Exercise 8, part (a)). Page faults = 9. (4 marks)
3. FIFO with 4 frames: Simulate:

Ref	1	2	3	4	1	2	5	1	2	3	4	5
F1	1*	1	1	1	1	1	5*	5	5	5	4*	4
F2		2*	2	2	2	2	2	1*	1	1	1	5*
F3			3*	3	3	3	3	3	2*	2	2	2
F4				4*	4	4	4	4	4	3*	3	3
Fault?	*	*	*	*			*	*	*	*	*	*

Page faults = 10. (4 marks)

4. Observation: Increasing frames from 3 to 4 increased page faults from 9 to 10. This is Belady's anomaly. (2 marks)

5. Algorithm: LRU (or any stack algorithm like Optimal). Stack algorithms satisfy the inclusion property: the set of pages in memory for n frames is always a subset of the set for $n + 1$ frames, so page faults never increase with more frames. (2 marks)

SOLUTION **Exercise 10 — Handling a page fault and thrashing — essay question**

1. Six steps of page fault handling:

   (a) The MMU detects an invalid page reference (valid bit = 0) and traps to the OS (page fault interrupt).

   (b) The OS checks the internal process table to determine if the reference is valid (within the process's address space). If invalid, terminate the process (segmentation fault).

   (c) If valid, the OS finds a free frame. If none, it selects a victim page using a replacement algorithm (e.g., LRU) and writes it to disk if dirty.

   (d) The OS schedules a disk read to bring the required page into the free frame.

   (e) Upon disk interrupt (I/O complete), the OS updates the page table: sets frame number, valid bit = 1, and resets other bits (e.g., reference, dirty).

   (f) The OS restarts the interrupted instruction (the CPU re-executes the instruction that caused the fault, now finding the page in memory).

   (6 marks)

2. Thrashing explanation: When many applications are open, each process's working set (the set of pages it actively uses) may exceed the available physical frames. The OS spends most of its time swapping pages in and out (high paging activity) rather than executing instructions. This is thrashing. Due to locality of reference, a process needs its working set resident to run efficiently; if the working set cannot fit, the pro-

cess constantly page-faults, causing the system to freeze. (4 marks)

3. Remedies:

   (a) Increase physical memory (add RAM): Larger memory can hold more working sets simultaneously, reducing page faults and thrashing. (Hardware solution.)

   (b) Adjust the degree of multiprogramming (swap out processes): The OS can suspend some processes (swap them out entirely) to free frames for the remaining processes, ensuring each active process has enough frames for its working set. (OS policy: working set model or page fault frequency.)

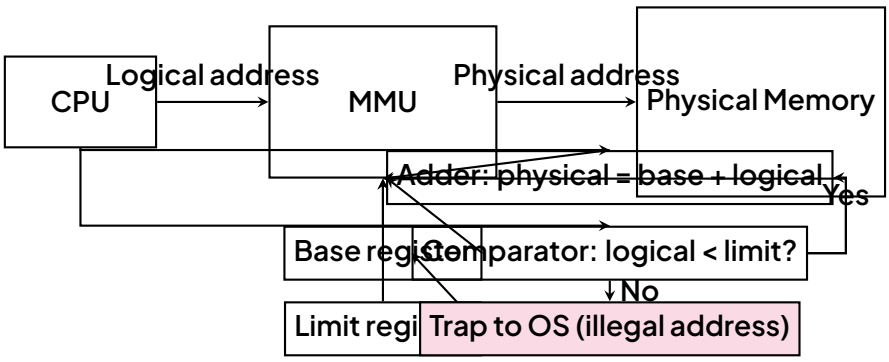
(4 marks)

SOLUTION **Exercise 11 — Address translation by the MMU — structured question**

1. Logical address: The address generated by the CPU (program's view). Physical address: The actual address in main memory hardware. Usefulness: Allows processes to have a contiguous logical address space starting at 0, independent of where they are loaded in physical memory. Enables memory protection, sharing, and relocation. (3 marks)

2. Base register: Holds the starting physical address of the process (relocation value). Every logical address is added to the base to get the physical address. Limit register: Holds the size (or maximum logical address) of the process. The MMU checks that the logical address < limit; if not, it traps (protection fault). Together they provide relocation and protection. (4 marks)

3. Diagram:



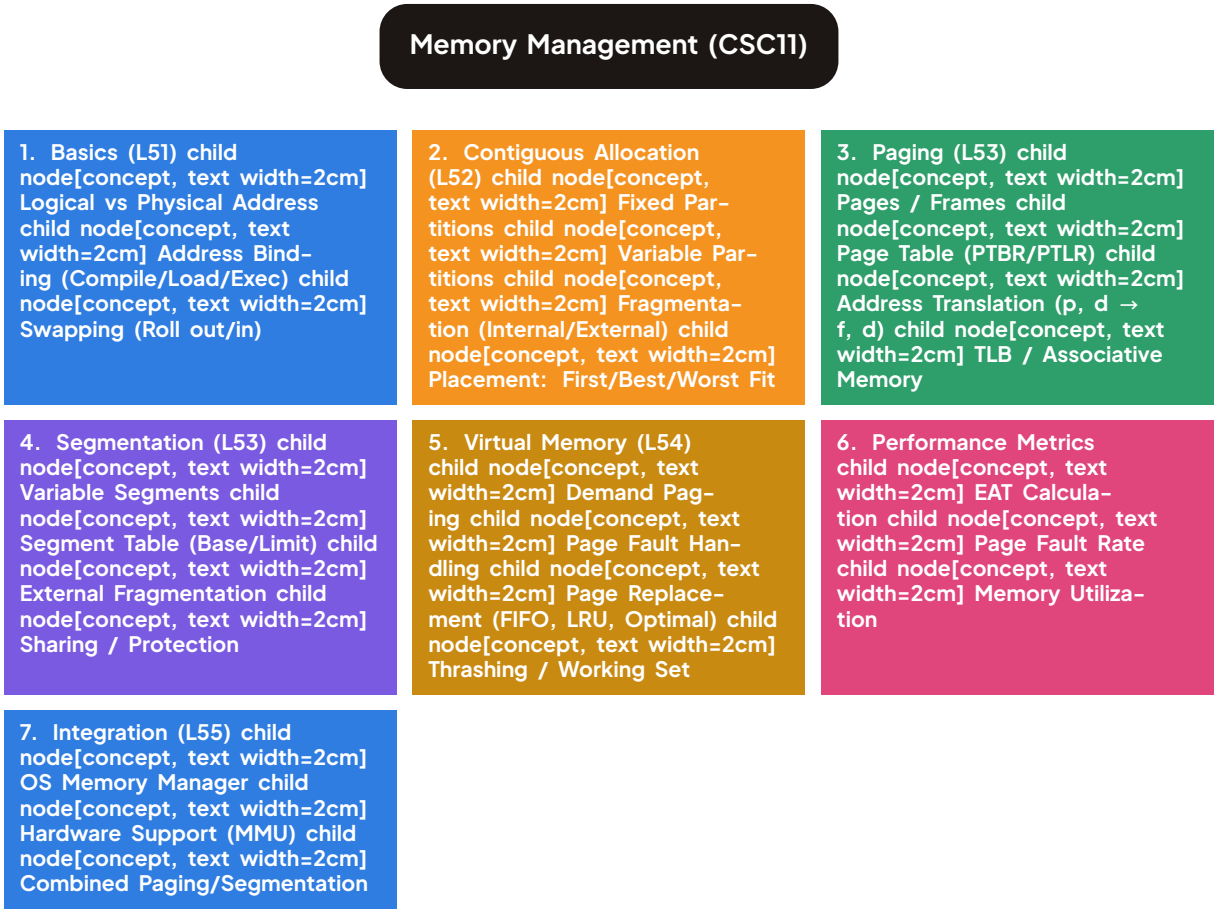
MMU address translation with base and limit registers

(4 marks)

4. Process loaded at physical base = 14000, size = 5000 bytes ⇒ valid logical addresses: 0 to 4999. Limit = 5000.
   - Logical 0: 0 < 5000 OK. Physical = 14000 + 0 = 14000.
   - Logical 2500: 2500 < 5000 OK. Physical = 14000 + 2500 = 16500.
   - Logical 4999: 4999 < 5000 OK. Physical = 14000 + 4999 = 18999.
   - Logical 5000: 5000 $\not<$ 5000 ⇒ limit exceeded. MMU traps to OS (protection fault), process terminated or signalled.

(4 marks)

13 SUMMARY SHEET



Chapter CSC11 Mind Map: Managing Memory Resources

FICHE BILAN **L'essentiel à connaître pour l'examen**

Key Definitions

- **Logical Address** (Virtual): Generated by CPU; **Physical Address**: Seen by memory unit.
- **Address Binding**: Mapping logical → physical. Compile time (absolute), Load time (relocatable), Execution time (dynamic, needs MMU).
- **Swapping**: Moving process between main memory and backing store (disk) to increase multiprogramming.
- **Fragmentation**: Internal (allocated > requested, fixed partitions); External (free holes too small, variable partitions/paging).

- **Paging**: Physical memory = **frames**, Logical memory = **pages** (same size). No external fragmentation.
- **Segmentation**: Logical view = variable-sized **segments** (code, data, stack). Supports sharing/protection.
- **Virtual Memory**: Separates user logical memory from physical memory; allows execution of partially loaded processes.
- **Demand Paging**: Load page only when needed (page fault). **Page Fault**: Reference to invalid page → trap to OS.
- **Thrashing**: Process spends more time paging than executing (high page fault rate, low CPU utilization).
- **Working Set**: Set of pages actively used by a process in a recent time window Δ.

Laws, Formulas & Theorems (Examinable)

- Effective Access Time (EAT):

$$\text{EAT} = (1 - p) \times t_{\text{mem}} + p \times t_{\text{page fault}}$$

  where p = page fault rate, t_{mem} = memory access time, $t_{\text{page fault}}$ = service time (disk I/O + restart).
- TLB Hit Ratio α: $\text{EAT}_{\text{TLB}} = \alpha(t_{\text{TLB}} + t_{\text{mem}}) + (1 - \alpha)(t_{\text{TLB}} + 2t_{\text{mem}})$.
- Paging Address Translation: Logical address $\langle p, d \rangle \rightarrow$ Frame f = PageTable$[p]$, Physical $\langle f, d \rangle$. Page size = $2^n \Rightarrow d = n$ LSBs.
- Segmentation Address Translation: Logical $\langle s, d \rangle \rightarrow$ Check d < limit$[s]$, Physical = base$[s] + d$.
- Belady's Anomaly: For FIFO replacement, increasing frames may increase page faults. Does not occur for LRU/Optimal (stack algorithms).

Methods (Step-by-Step)

1. Contiguous Placement (First/Best/Worst Fit): Scan partition list; allocate first/ smallest/ largest hole ≥ process size. Update hole list (split/merge).
2. Page Fault Handling:
   (a) Trap to OS (save PC/regs).
   (b) Check validity (valid bit in PCB/Page Table).
   (c) Find free frame (or select victim via replacement algo).
   (d) Schedule disk read (block process).
   (e) On interrupt: update Page Table, set valid bit.
   (f) Restart instruction.

3. Page Replacement Trace (FIFO/LRU/Optimal): Draw frame columns; fill on fault; apply policy on full frames; count faults.
4. EAT Calculation: Identify p, t_{mem}, t_{fault}; plug into formula; watch units (ms vs ns).

Common Mistakes (Traps)

- Confusing **page** (logical) with **frame** (physical). They are same size but different locations.
- Thinking paging eliminates all fragmentation (internal fragmentation exists in last page).
- Forgetting **valid/invalid bit** in page table entry (causes page fault).
- Misidentifying **external fragmentation** in segmentation vs paging.
- In EAT: using t_{fault} as only disk time, ignoring memory access to update tables/restart.
- Belady's Anomaly: claiming it happens for LRU (it does not).
- Swapping vs Paging: Swapping moves whole process; Paging moves individual pages.

GCE Advanced Level Tips

- Trace Tables are King: Always draw a table for placement (First/Best/Worst Fit) and replacement (FIFO/LRU/Optimal) questions. Show frame contents after each reference.
- Address Translation Diagrams: Be ready to draw the CPU → MMU → Page Table → Physical Memory flow. Label bits (p, d, f).
- Definitions: Learn verbatim: "Virtual memory allows execution of processes not completely in memory", "Thrashing is high paging activity".
- Calculation Questions: Convert units carefully (1 ms = 10^6 ns). EAT questions often give page fault service time in ms and memory access in ns.
- Comparison Essays: "Compare Paging and Segmentation" → Table: Size (Fixed/Variable), Fragmentation (Internal/External), Address Structure, Sharing, Hardware Support.
- Integration (L55): Expect a scenario: "A system has 64MB RAM, 4KB pages... calculate page table size / EAT / page faults for given string". Combine all concepts.

Signed • OnBuch+
ONBUCH+ TEACHING TEAM
FOLLOWS THE OFFICIAL MINESEC SYLLABUS

OnBuch+